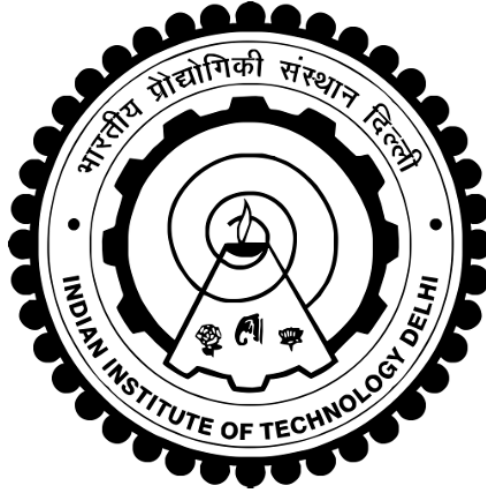


# P1 Requirements + Specifications + Design

Tribe: Tuesday Tribe

March 13, 2026



## Abstract

This document describes the design and fabrication of a versatile function generator (FG) using the Espressif Systems 32 microcontroller. The FG generates sine, square, and triangle waves up to 1 MHz, with basic AM and FM modulation. The design focuses on practicality, offering a valuable tool for students, hobbyists, and small-scale developers. Using readily available components, the project aims to provide a fully functional Function Generator (FG) suitable for first-year electrical engineering students in academic laboratory environments.

## Team Members

| Sl. | Name                       | Entry No. | Designation                   | IF  |
|-----|----------------------------|-----------|-------------------------------|-----|
| 1   | Agrim Chandra              | MT1221952 | Documentation Team Member     | 0.7 |
| 2   | Vulasala Jayanth           | MT1230687 | Hardware Team Member          | 0.7 |
| 3   | Ayan Goswami               | MT1230159 | Hardware Team Member          | 0.8 |
| 4   | Agnik Halder               | EE3230778 | Hardware Team Member          | 0.9 |
| 5   | Jayesh Raj Singhal         | EE3230869 | Hardware Team Member          | 0.9 |
| 6   | Kartik Vaishnav            | MT6231064 | Hardware Team Member          | 0.9 |
| 7   | Aashay Saxena              | EE3230963 | Software Team Member          | 1   |
| 8   | Aditeya Srivastava         | EE1230527 | Hardware Sub-coordinator      | 1   |
| 9   | Aditya Jaiswal             | EE1230512 | Tribe Coordinator             | 1   |
| 10  | Aditya Kumar               | EE1230339 | Software Team Member          | 1   |
| 11  | Akshat Gupta               | EE1230572 | Documentation Team Member     | 1   |
| 12  | Amit Jaiswal               | EE1230605 | Hardware Team Member          | 1   |
| 13  | Archie Mahajan             | EE3230748 | Documentation Sub-coordinator | 1   |
| 14  | Archit Singla              | EE3230516 | Documentation Team Member     | 1   |
| 15  | Arnab Goyal                | MT6221963 | Software Team Member          | 1   |
| 16  | Arpan Jain                 | EE1230700 | Deputy Tribe Coordinator      | 1   |
| 17  | Asteek Narayan             | MT1230137 | Software Team Member          | 1   |
| 18  | Atharva Prashant Patil     | EE3230754 | Hardware Team Member          | 1   |
| 19  | Ayaan Alam                 | MT1230215 | Hardware Team Member          | 1   |
| 20  | Ayush Bharat Panchgade     | MT1230569 | Hardware Team Member          | 1   |
| 21  | Ayush Goyal                | MT1230832 | Documentation Team Member     | 1   |
| 22  | BANAVATHU JATHIN AMAR Naik | MT6230997 | Software Team Member          | 1   |
| 23  | CHAPPIDI MANI-PAL REDDY    | EE1190472 | Hardware Team Member          | 1   |
| 24  | Dhruv                      | EE3230375 | Hardware Team Member          | 1   |
| 25  | Hardik Agarwal             | EE1230661 | Hardware Sub-coordinator      | 1   |
| 26  | Harshik Bains              | EE1230425 | Documentation Team Member     | 1   |
| 27  | Harshul Anand              | EE1230798 | Software Team Member          | 1   |

| Sl. | Name                                   | Entry No. | Designation               | IF |
|-----|----------------------------------------|-----------|---------------------------|----|
| 28  | Harshvi Kanwatiya                      | EE3230341 | Documentation Team Member | 1  |
| 29  | Ishaan Singh                           | EE1230784 | Documentation Team Member | 1  |
| 30  | Ishan Arya                             | EE1230379 | Software Team Member      | 1  |
| 31  | Janhavi Sahu                           | EE1230327 | Documentation Team Member | 1  |
| 32  | Jatin Jindal                           | EE3230530 | Hardware Team Member      | 1  |
| 33  | Joy sujeeth dan-<br>dolu               | MT1230444 | Software Team Member      | 1  |
| 34  | Kalavakuntla Anil<br>Srinivas          | EE1230575 | Hardware Team Member      | 1  |
| 35  | Kalpesh Meena                          | EE1230394 | Hardware Sub-coordinator  | 1  |
| 36  | Khushank Galgat                        | MT1221291 | Software Team Member      | 1  |
| 37  | Khushank Verma                         | MT1231199 | Software Team Member      | 1  |
| 38  | Kowshik Naidu<br>Tentu                 | EE3230511 | Hardware Team Member      | 1  |
| 39  | Krish Goel                             | EE3230113 | Software Sub-coordinator  | 1  |
| 40  | Krishna Prajapat                       | MT1230515 | Hardware Team Member      | 1  |
| 41  | Lakshmipuram<br>Charan Karthik         | EE1230481 | Hardware Team Member      | 1  |
| 42  | Lalam Sai Jahnavi                      | EE1230780 | Documentation Team Member | 1  |
| 43  | MADDALA<br>VEERA<br>VENKATA MO-<br>HAN | EE1230558 | Hardware Team Member      | 1  |
| 44  | Manas Agrawal                          | EE1230753 | Documentation Team Member | 1  |
| 45  | Manashvi Garg                          | EE1231104 | Hardware Sub-coordinator  | 1  |
| 46  | Mayank Singal                          | EE1231171 | Documentation Team Member | 1  |
| 47  | Md Abubakar                            | EE3230400 | Hardware Team Member      | 1  |
| 48  | Md Ashhadur Rah-<br>man                | MT1230908 | Software Team Member      | 1  |
| 49  | Naitik Singla                          | EE1230519 | Hardware Team Member      | 1  |
| 50  | Nandini Chanana                        | EE3230669 | Hardware Sub-coordinator  | 1  |
| 51  | Nepurn Tyagi                           | MT1230092 | Software Team Member      | 1  |
| 52  | Nirmal Rewar                           | EE3231187 | Software Team Member      | 1  |
| 53  | Piyush Abnave                          | MT1221289 | Software Team Member      | 1  |

| Sl. | Name                  | Entry No. | Designation                   | IF |
|-----|-----------------------|-----------|-------------------------------|----|
| 54  | Prashant Singh        | EE3230829 | Documentation Team Member     | 1  |
| 55  | Praveen Panwar        | MT1231168 | Documentation Team Member     | 1  |
| 56  | Prerak Singh          | EE1230898 | Documentation Team Member     | 1  |
| 57  | Prikshit Khyalia      | MT1230513 | Software Team Member          | 1  |
| 58  | Raghav Modi           | EE3231203 | Hardware Team Member          | 1  |
| 59  | Rakshit Choudhary     | EE3230645 | Hardware Team Member          | 1  |
| 60  | Ravi Chauhan          | MT1230395 | Software Team Member          | 1  |
| 61  | Ridhi Maheshwari      | MT1230839 | Hardware Team Member          | 1  |
| 62  | Rishit Ojha           | MT1230332 | Documentation Team Member     | 1  |
| 63  | Samarth Tiwari        | EE1230550 | Documentation Team Member     | 1  |
| 64  | Sanidhya Ojha         | EE3230694 | Hardware Sub-coordinator      | 1  |
| 65  | Sanyam Garg           | MT1230732 | Hardware Team Member          | 1  |
| 66  | Shambhavi Malviya     | EE3230455 | Hardware Team Member          | 1  |
| 67  | Shashwat Patel        | MT1230274 | Software Sub-coordinator      | 1  |
| 68  | Shriram Chandra Sahoo | EE1230720 | Documentation Team Member     | 1  |
| 69  | Shubhankur Tri-pathi  | EE1231085 | Hardware Team Member          | 1  |
| 70  | Siya Jain             | MT1230373 | Hardware Team Member          | 1  |
| 71  | Smart Yadav           | EE1230412 | Hardware Team Member          | 1  |
| 72  | Soham Sameer Palkar   | MT2261971 | Software Team Member          | 1  |
| 73  | Suchir Kalra          | EE1230681 | Hardware Sub-coordinator      | 1  |
| 74  | Swarali Pawar         | EE3230475 | Documentation Sub-coordinator | 1  |
| 75  | Tamanna Thakral       | EE1230747 | Hardware Team Member          | 1  |
| 76  | Tanmay Gupta          | EE3230797 | Hardware Team Member          | 1  |
| 77  | Tushar Kumar Sinha    | MT6230608 | Hardware Team Member          | 1  |
| 78  | Urva Madan            | MT1230480 | Hardware Team Member          | 1  |
| 79  | Vaibhav Sharma        | MT1230146 | Deputy Tribe Coordinator      | 1  |
| 80  | Vaidik Gupta          | EE3221749 | Software Team Member          | 1  |
| 81  | Vansh Saini           | EE1230656 | Documentation Sub-coordinator | 1  |
| 82  | Vanshika Bathla       | EE3230828 | Documentation Team Member     | 1  |

| <b>Sl.</b> | <b>Name</b>      | <b>Entry No.</b> | <b>Designation</b>        | <b>IF</b> |
|------------|------------------|------------------|---------------------------|-----------|
| 83         | Velamala Bhavana | MT6230210        | Software Team Member      | 1         |
| 84         | Vrinda Bansal    | EE1230825        | Documentation Team Member | 1         |
| 85         | Yash Kumar       | EE3231196        | Software Team Member      | 1         |
| 86         | Yash Garg        | EE3230587        | Hardware Team Member      | 1         |
| 87         | Yuv Verdia       | MT1230770        | Documentation Team Member | 1         |

# Contents

|                                                |           |
|------------------------------------------------|-----------|
| <b>Glossary</b>                                | <b>10</b> |
| <b>1 Requirements</b>                          | <b>11</b> |
| 1.1 Display Requirement . . . . .              | 11        |
| 1.1.1 Display Type . . . . .                   | 11        |
| 1.1.2 Functional Usage . . . . .               | 11        |
| 1.2 Output Requirement . . . . .               | 11        |
| 1.2.1 Output Voltage Characteristics . . . . . | 11        |
| 1.2.2 Frequency Specifications . . . . .       | 11        |
| 1.2.3 Supported Output Waveforms . . . . .     | 11        |
| 1.3 Input Requirement . . . . .                | 11        |
| 1.3.1 Power Supply . . . . .                   | 11        |
| 1.3.2 Input Components . . . . .               | 11        |
| 1.4 Wave IC Requirement . . . . .              | 12        |
| 1.4.1 IC Description . . . . .                 | 12        |
| 1.4.2 Frequency Performance . . . . .          | 12        |
| 1.4.3 Waveform Capability . . . . .            | 12        |
| 1.4.4 Communication Protocol . . . . .         | 12        |
| 1.4.5 Advantages . . . . .                     | 12        |
| 1.5 Control Interface Requirement . . . . .    | 12        |
| 1.5.1 User Input Devices . . . . .             | 12        |
| 1.6 Microcontroller Requirement . . . . .      | 12        |
| 1.6.1 Controller Selection . . . . .           | 12        |
| 1.7 Additional Hardware Requirement . . . . .  | 13        |
| 1.7.1 Output Connectivity . . . . .            | 13        |
| 1.7.2 Dual Channel Functionality . . . . .     | 13        |
| 1.8 Communication Method . . . . .             | 13        |
| 1.8.1 Wi-Fi Based Control . . . . .            | 13        |
| 1.8.2 Technical Stack . . . . .                | 13        |
| 1.9 Software Requirements . . . . .            | 13        |
| 1.9.1 Programming Environment . . . . .        | 13        |
| 1.9.2 ESP32 Firmware Requirements . . . . .    | 13        |
| 1.9.3 Communication Software . . . . .         | 14        |
| 1.9.4 Software Advantages . . . . .            | 14        |
| <b>2 Mind Map and Workflow Diagrams</b>        | <b>14</b> |
| 2.1 Project Timeline . . . . .                 | 18        |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>3</b> | <b>Specifications</b>                                      | <b>18</b> |
| 3.1      | Front Panel Layouts and Controls . . . . .                 | 18        |
| 3.1.1    | Frequency Controls . . . . .                               | 18        |
| 3.1.2    | Cursor-Based Digit Editing . . . . .                       | 18        |
| 3.1.3    | Frequency Resolution and Units . . . . .                   | 19        |
| 3.1.4    | Amplitude Control . . . . .                                | 19        |
| 3.1.5    | DC Offset Control . . . . .                                | 19        |
| 3.1.6    | Display and Indicators . . . . .                           | 20        |
| 3.1.7    | Output Controls . . . . .                                  | 20        |
| 3.1.8    | Front Panel Layout Specifications . . . . .                | 21        |
| 3.2      | Core Components . . . . .                                  | 23        |
| 3.2.1    | ESP32 Microcontroller . . . . .                            | 23        |
| 3.2.2    | AD9833 Function Generator IC . . . . .                     | 23        |
| 3.2.3    | 1.3-inch OLED Display Module . . . . .                     | 24        |
| 3.2.4    | Operational Amplifiers . . . . .                           | 24        |
| 3.3      | Power Specification . . . . .                              | 24        |
| 3.3.1    | Power Input . . . . .                                      | 24        |
| 3.3.2    | Output Signal Specifications . . . . .                     | 24        |
| 3.3.3    | Environmental Power Considerations . . . . .               | 24        |
| 3.4      | App Innovations . . . . .                                  | 25        |
| 3.4.1    | Software-Defined "Safe Limits" (Protection Mode) . . . . . | 25        |
| 3.4.2    | Automated Frequency Sweep for Testing . . . . .            | 25        |
| 3.4.3    | Component Preset Library . . . . .                         | 25        |
| 3.4.4    | Non-Volatile Memory (NVM) Recall . . . . .                 | 25        |
| 3.4.5    | Firmware-over-the-Air (FOTA) . . . . .                     | 25        |
| 3.5      | Bill of Materials . . . . .                                | 26        |
| <b>4</b> | <b>Design</b>                                              | <b>27</b> |
| 4.1      | CAD Design . . . . .                                       | 27        |
| 4.2      | Prototype Implementation . . . . .                         | 28        |
| 4.3      | CAD Diagrams . . . . .                                     | 31        |
| 4.4      | Testing of CAD Design . . . . .                            | 32        |
| 4.5      | Display Interface Design . . . . .                         | 32        |
| 4.6      | Hardware Design . . . . .                                  | 33        |
| 4.7      | Front Panel Usage . . . . .                                | 34        |
| 4.8      | Firmware and Embedded Code . . . . .                       | 36        |
| 4.8.1    | Firmware Directory Structure . . . . .                     | 36        |
| 4.8.2    | Application Module . . . . .                               | 36        |
| 4.8.3    | Display Module . . . . .                                   | 40        |

|       |                                         |    |
|-------|-----------------------------------------|----|
| 4.8.4 | Hardware Drivers . . . . .              | 40 |
| 4.8.5 | Input Handling . . . . .                | 42 |
| 4.8.6 | Signal Generation . . . . .             | 43 |
| 4.9   | Web-Based Control Interface . . . . .   | 44 |
| 4.9.1 | Web Application Architecture . . . . .  | 44 |
| 4.9.2 | Web Interface Implementation . . . . .  | 44 |
| 4.9.3 | Web Server Backend . . . . .            | 63 |
| 4.10  | Techniques and Implementation . . . . . | 74 |

|                           |           |
|---------------------------|-----------|
| <b>Minutes of Meeting</b> | <b>75</b> |
|---------------------------|-----------|

## List of Tables

|   |                                                        |    |
|---|--------------------------------------------------------|----|
| 2 | Component Cost Table . . . . .                         | 26 |
| 3 | Techniques used during prototype development . . . . . | 74 |
| 5 | Hardware Component Cost Estimates . . . . .            | 96 |

## List of Figures

|    |                                                                                                                                                                                                                                                                                           |    |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Consolidated Software Team Flowchart . . . . .                                                                                                                                                                                                                                            | 15 |
| 2  | Project Mind Map (TikZ/Forest) . . . . .                                                                                                                                                                                                                                                  | 16 |
| 3  | Workflow Diagram for Research Team . . . . .                                                                                                                                                                                                                                              | 17 |
| 4  | Workflow Diagram Hardware Team . . . . .                                                                                                                                                                                                                                                  | 17 |
| 5  | Gantt chart depicting the project timeline from January 13 to February 20, 2026. The chart illustrates four phases: initial analysis and ideation (Phase 1), design and simulation work (Phase 2), prototyping activities (Phase 3), and final production and assembly (Phase 4). . . . . | 18 |
| 6  | Schematic Diagram of Function Generator Design (Side View) . . . . .                                                                                                                                                                                                                      | 21 |
| 7  | Schematic Diagram of Function Generator Design (Front View) . . . . .                                                                                                                                                                                                                     | 21 |
| 8  | Schematic Diagram of Function Generator Design (Top View) . . . . .                                                                                                                                                                                                                       | 22 |
| 9  | CAD model of the prototype enclosure . . . . .                                                                                                                                                                                                                                            | 28 |
| 10 | Internal electronics and wiring inside the prototype . . . . .                                                                                                                                                                                                                            | 29 |
| 11 | Prototype acrylic enclosure of the function generator . . . . .                                                                                                                                                                                                                           | 30 |
| 12 | Front panel layout showing keypad, display and controls . . . . .                                                                                                                                                                                                                         | 30 |
| 13 | Initial hardware testing of the OLED display module connected to the ESP32 microcontroller on a breadboard. The screen shows the generated waveform and operating parameters during early firmware validation. . . . .                                                                    | 31 |
| 14 | Front, top, bottom and side views of the function generator. . . . .                                                                                                                                                                                                                      | 32 |
| 15 | Circuit diagram of the waveform generation system (placeholder) . . . . .                                                                                                                                                                                                                 | 34 |

|    |                                                                                                                             |     |
|----|-----------------------------------------------------------------------------------------------------------------------------|-----|
| 16 | Block diagram of the waveform generation hardware architecture . . . . .                                                    | 34  |
| 17 | Annotated front panel layout showing keypad, OLED display, control buttons, potentiometers, and cable routing slot. . . . . | 35  |
| 18 | Bottom section of the front panel showing the power switch and BNC output connectors. . . . .                               | 35  |
| 19 | Electrical connections for the front panel components . . . . .                                                             | 36  |
| 20 | Preliminary System Architecture . . . . .                                                                                   | 84  |
| 21 | Thermocol prototype used to evaluate front panel wedge angles . . . . .                                                     | 98  |
| 22 | Initial discussion and sketches on enclosure dimensions . . . . .                                                           | 99  |
| 23 | Prototype acrylic enclosure for the function generator . . . . .                                                            | 105 |

# Glossary

|                         |                                                                                                                                                                                          |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AD9833</b>           | Low-cost Direct Digital Synthesis (DDS) IC that generates sine, triangle and square waveforms. External filtering and amplification are typically required for larger output amplitudes. |
| <b>ADC/DAC</b>          | ADC: <i>Analog-to-Digital Converter</i> ; DAC: <i>Digital-to-Analog Converter</i> ; used to convert between analog voltages and digital values                                           |
| <b>DDS</b>              | <i>Direct Digital Synthesis</i> - digital technique that generates waveforms using a phase accumulator and lookup or computation, used by the AD9833.                                    |
| <b>ESP32</b>            | Espressif Systems microcontroller family with integrated Wi-Fi and Bluetooth; used here as the central system controller.                                                                |
| <b>OLED</b>             | <i>Organic Light-Emitting Diode</i> display (1.3" monochrome OLED used in this project).                                                                                                 |
| <b>BNC</b>              | <i>Bayonet Neill-Concelman</i> coaxial connector used for signal outputs and measurement connections.                                                                                    |
| <b>SPI</b>              | <i>Serial Peripheral Interface</i> - synchronous serial bus used to control the AD9833 and other SPI devices.                                                                            |
| <b>I<sup>2</sup>C</b>   | <i>Inter-Integrated Circuit</i> - two-wire serial bus used by displays and sensors (often written I2C or I <sup>2</sup> C).                                                              |
| <b>OTA</b>              | <i>Over-The-Air</i> firmware update method (delivered via Wi-Fi) for updating microcontroller firmware without physical access.                                                          |
| <b>PCB</b>              | <i>Printed Circuit Board</i> - the board used to mount components and route signals.                                                                                                     |
| <b>BoM</b>              | <i>Bill of Materials</i> - list of components and costs used for project planning and procurement (appears as "Bill of Materials" / Component Cost Table).                               |
| <b>Buck/Boost</b>       | Power-conversion topologies commonly used in the power supply stage to step voltage down (buck) or up (boost).                                                                           |
| <b>Banana connector</b> | Standard DC/power connector (banana pin) used in bench power supplies and mentioned under power/connectivity notes.                                                                      |
| <b>OPA810 / LM7171</b>  | High-speed operational amplifiers cited for signal conditioning, buffering, amplification, and DC offset generation in the analog output stage.                                          |
| <b>U8g2</b>             | Graphics library used for rendering on small OLED displays (the report references U8g2 in Full Frame mode).                                                                              |
| <b>Websocket</b>        | Full-duplex web communication protocol (used for real-time control in web-based interfaces).                                                                                             |
| <b>REST API</b>         | Representational State Transfer - HTTP-based interface style used for request/response interactions in web services                                                                      |
| <b>JSON</b>             | <i>JavaScript Object Notation</i> - lightweight data-interchange format used for web-control messages.                                                                                   |
| <b>DAC resolution</b>   | (concept) Higher DAC resolution (more bits) improves waveform fidelity - discussed in the context of ESP32 internal DAC vs external DDS.                                                 |

# 1 Requirements

## 1.1 Display Requirement

### 1.1.1 Display Type

- **OLED display** as the primary **visual interface** for user feedback.
- **1.3-inch OLED display** with **SPI and I2C communication** support for compatibility.

### 1.1.2 Functional Usage

- Display of key **system parameters** such as **waveform type** and **frequency**.

## 1.2 Output Requirement

### 1.2.1 Output Voltage Characteristics

- Maximum output amplitude: **20 V peak-to-peak**.

### 1.2.2 Frequency Specifications

- Output frequency range: **2 Hz to 2 MHz**.
- Frequency resolution: **0.004 Hz**.

### 1.2.3 Supported Output Waveforms

- Supported waveforms: **square, triangular, sine, half square**.

## 1.3 Input Requirement

### 1.3.1 Power Supply

- DC input supply:  **$\pm 12$  V**.

### 1.3.2 Input Components

- **Rotary Knobs** and **Numpad** for control.
- **Display screen** for user interaction.

## 1.4 Wave IC Requirement

### 1.4.1 IC Description

- **AD9833 waveform generator IC** for waveform synthesis.
- **Direct Digital Synthesis (DDS)** based operation.

### 1.4.2 Frequency Performance

- Clean output range: **0–1 MHz**.
- Higher frequency operation possible with potential distortion.

### 1.4.3 Waveform Capability

- Waveforms supported by IC: **sine, triangle, square, half-square** .

### 1.4.4 Communication Protocol

- Communication protocol: **SPI**.

### 1.4.5 Advantages

- Widely available.
- No support for arbitrary waveform generation.
- Low output amplitude.
- External filter and amplifier required.

## 1.5 Control Interface Requirement

### 1.5.1 User Input Devices

- Knobs for manual adjustment.
- Numpad for numerical input.
- Rotary knobs for smooth control.
- Shortcut button for switching between preset commonly used frequencies in ELP1000

## 1.6 Microcontroller Requirement

### 1.6.1 Controller Selection

- **ESP32** microcontroller.

## 1.7 Additional Hardware Requirement

### 1.7.1 Output Connectivity

- **BNC female coaxial connector** for output connection.

### 1.7.2 Dual Channel Functionality

- User can take output from **two different channels** for their experiments.

## 1.8 Communication Method

### 1.8.1 Wi-Fi Based Control

- Local web server hosted on ESP32.
- No separate mobile application required, Browser-based control interface.
- Cross-platform compatibility (Android, iOS, desktop).

**Bluetooth-Based Control** with dedicated application shall be required.

### 1.8.2 Technical Stack

- **Programming Language:** C / C++ using Aurdino IDE.
- **ESP32 Variant:** ESP32-WROOM-32

## 1.9 Software Requirements

### 1.9.1 Programming Environment

- The firmware shall be developed in **C / C++**.
- The **Arduino IDE** shall be used for compilation and uploading to the ESP32.

### 1.9.2 ESP32 Firmware Requirements

- The ESP32 shall establish a connection to Wi-Fi.
- The ESP32 shall update the frequency and waveform in real-time via recieved user commands.

### **1.9.3 Communication Software**

- A web page interface shall be provided for system control.
- The software shall allow for waveform selection.
- The system shall calculate the correct frequency tuning words.
- The interface shall allow control of:
  - Frequency
  - Waveform type
  - Output ON/OFF

### **1.9.4 Software Advantages**

- Device control shall be accessible via any web browser, providing real-time control.
- The architecture shall allow for future feature expansion.

The development workflow, task distribution, and timeline for the software team to meet these requirements are illustrated in Figure 1.

## **2 Mind Map and Workflow Diagrams**

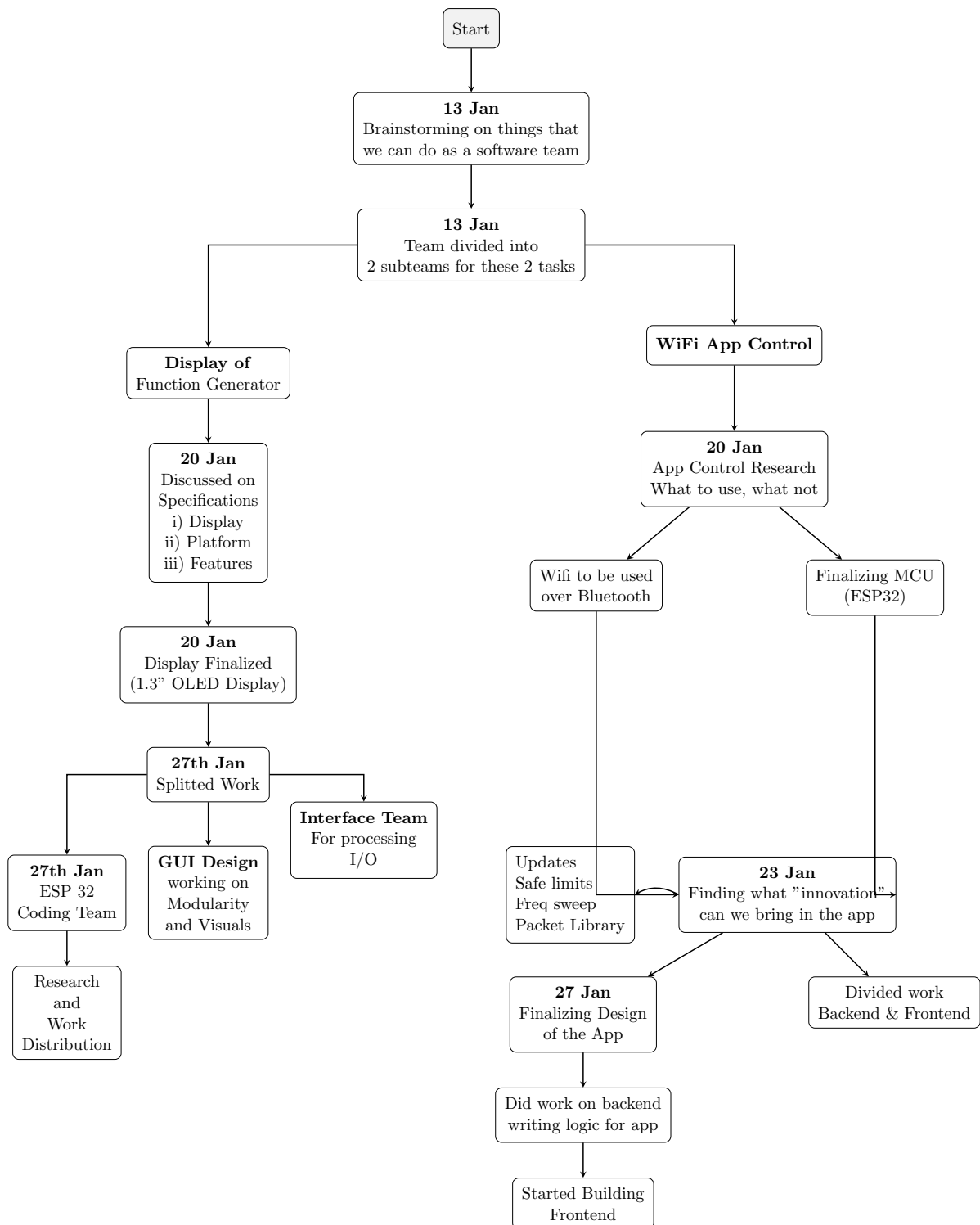


Figure 1: Consolidated Software Team Flowchart

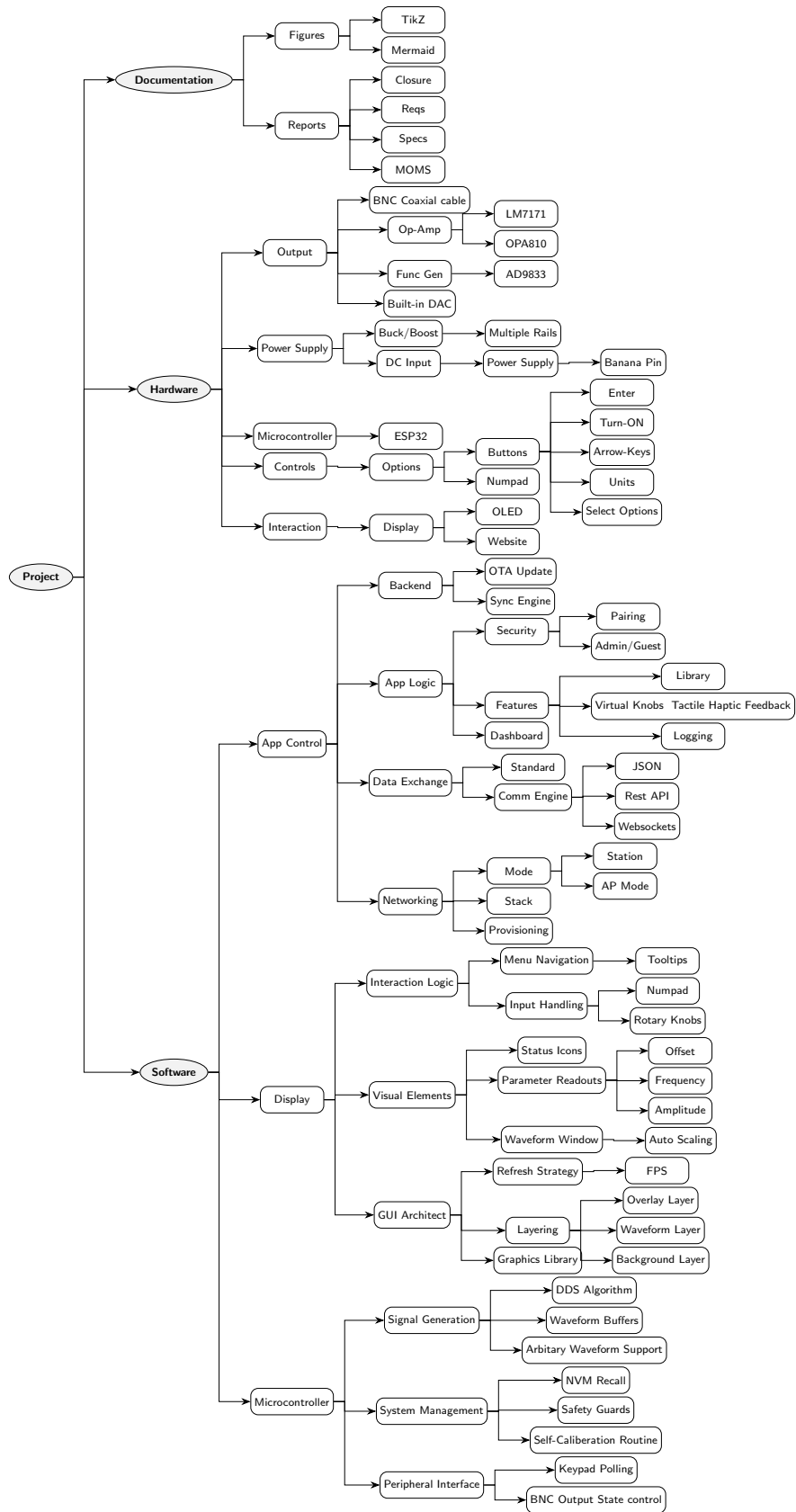


Figure 2: Project Mind Map (TikZ/Forest)

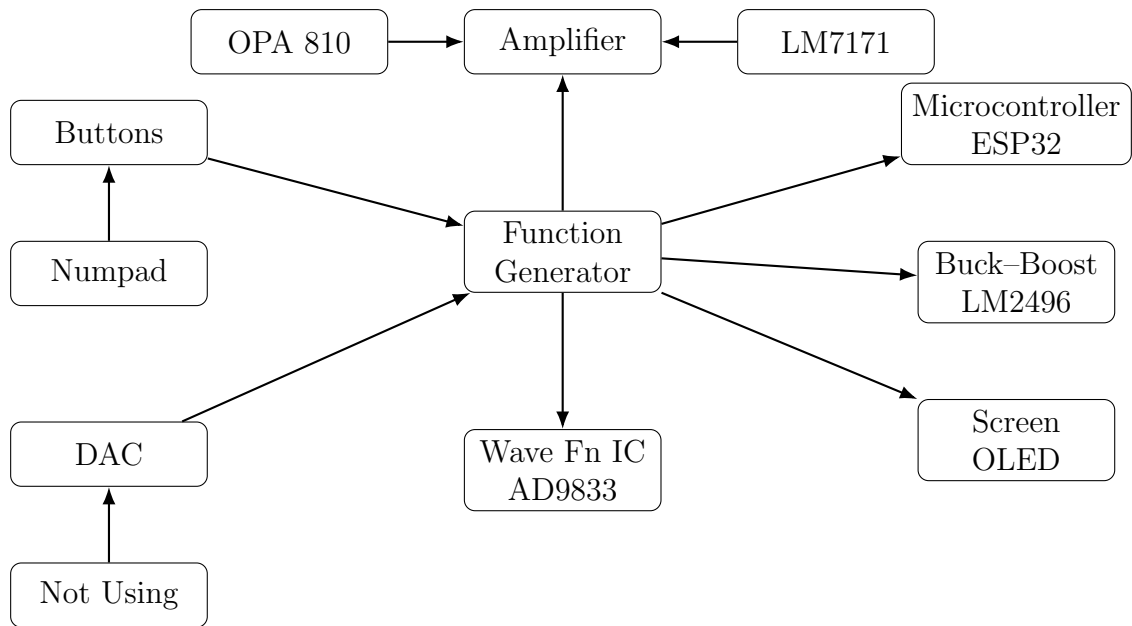


Figure 3: Workflow Diagram for Research Team

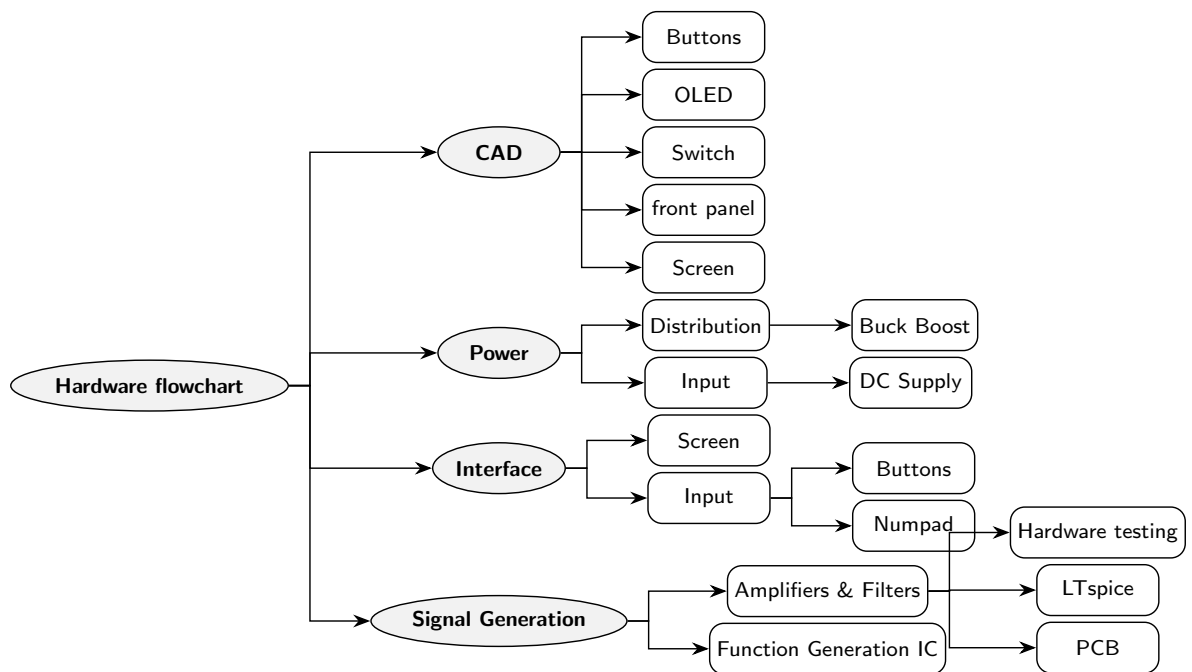


Figure 4: Workflow Diagram Hardware Team

## 2.1 Project Timeline

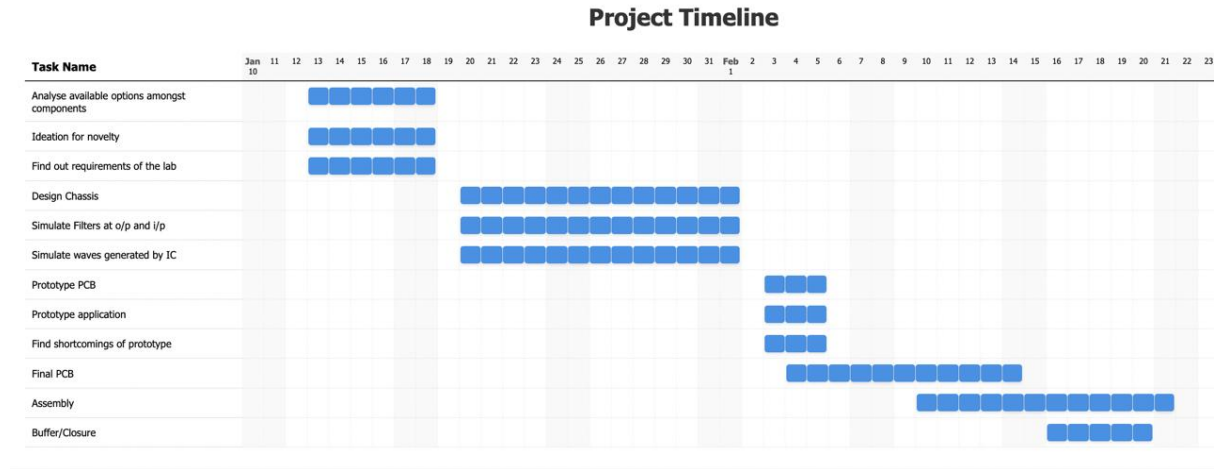


Figure 5: Gantt chart depicting the project timeline from January 13 to February 20, 2026. The chart illustrates four phases: initial analysis and ideation (Phase 1), design and simulation work (Phase 2), prototyping activities (Phase 3), and final production and assembly (Phase 4).

## 3 Specifications

### 3.1 Front Panel Layouts and Controls

#### 3.1.1 Frequency Controls

Frequency adjustment is performed using a **20-key numeric keypad**, consisting of:

- **16 numeric keys (0–9 and auxiliary numeric inputs)**
- **2 function keys**
- **2 dedicated cursor control keys**

The frequency value is displayed numerically on the OLED screen and is editable up to the **second decimal place**. A **cursor-based editing mechanism** is used to modify individual digits of the frequency value.

#### 3.1.2 Cursor-Based Digit Editing

Two dedicated cursor keys on the numpad enable precise navigation across the displayed frequency value:

- The **left cursor key** moves the cursor to the previous digit.

- The **right cursor key** moves the cursor to the next digit.

The currently selected digit is highlighted on the display. Numeric keys are used to modify only the selected digit, allowing fine-grained control without re-entering the entire frequency value.

### 3.1.3 Frequency Resolution and Units

The system supports frequency adjustment with a resolution up to **two decimal places**. Frequency values can be entered and displayed in multiple units, including:

- mHz
- Hz
- kHz
- MHz

Unit selection and value modification are both handled through the numeric keypad in conjunction with the cursor controls.

### 3.1.4 Amplitude Control

- **Dedicated Potentiometer:**
  - Adjustable range: 0 V to 20 V peak-to-peak.
- **Fine/Coarse Adjustment Switch:**
  - Fine: Enables precise amplitude adjustment.
  - Coarse: Allows rapid amplitude changes.

### 3.1.5 DC Offset Control

- **Offset Potentiometer:**
  - Adjustable range:  $\pm 12$  V
  - Center detent at 0 V for easy reference
  - Calibrated scale markings for precise tuning
- **Offset ON/OFF Switch:**
  - Enables or disables DC offset
  - LED indicator glows when offset is active

### 3.1.6 Display and Indicators

- **1.3-inch OLED Display:**
  - **OLED Display Module:**
    - \* 1.3-inch monochrome OLED display
    - \* 4-pin interface configuration
    - \* Operates at 3.3 V DC input supply
  - **Communication Interface:**
    - \* I<sup>2</sup>C (IIC) communication protocol
    - \* Operating frequency of 400 kHz
    - \* Receives display data from the microcontroller
  - **Driver and Rendering:**
    - \* U8g2 graphics library in Full Frame mode
    - \* Optimized for fast and efficient rendering
  - **Visual Features:**
    - \* Automatic frequency unit scaling (Hz / kHz)
    - \* Toggle capability between two output channels
  - **Footer Indicators:**
    - \* Waveform type icons (Sine, Square, Triangle)
    - \* Output ON/OFF status indication

### 3.1.7 Output Controls

- Output power switch
- Ground terminal
- Dual BNC output connectors(CH1 and CH2)
- Selectable waveforms (via menu):
  - Sine
  - Triangle
  - Square
  - Half-Square

### 3.1.8 Front Panel Layout Specifications

- **Dimensions:**

- Width  $\times$  Height  $\times$  Depth: 200 mm  $\times$  80 mm  $\times$  240 mm

- Knob Port diameter (DC Offset, Amplitude, and Output): 11 mm
- Extruded front panel thickness: 5 mm
- Power supply/input dimensions: 28.8 mm  $\times$  26 mm
- Main Display Cutout: 64 mm  $\times$  64 mm
- Push button dimensions (Diameter  $\times$  Thickness): 10 mm  $\times$  1 mm

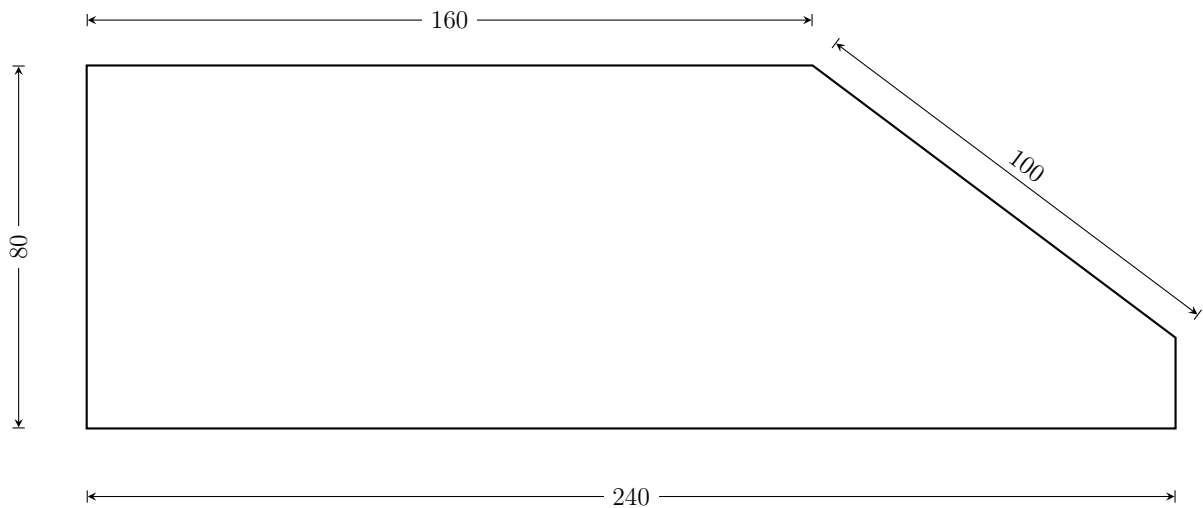


Figure 6: Schematic Diagram of Function Generator Design (Side View)

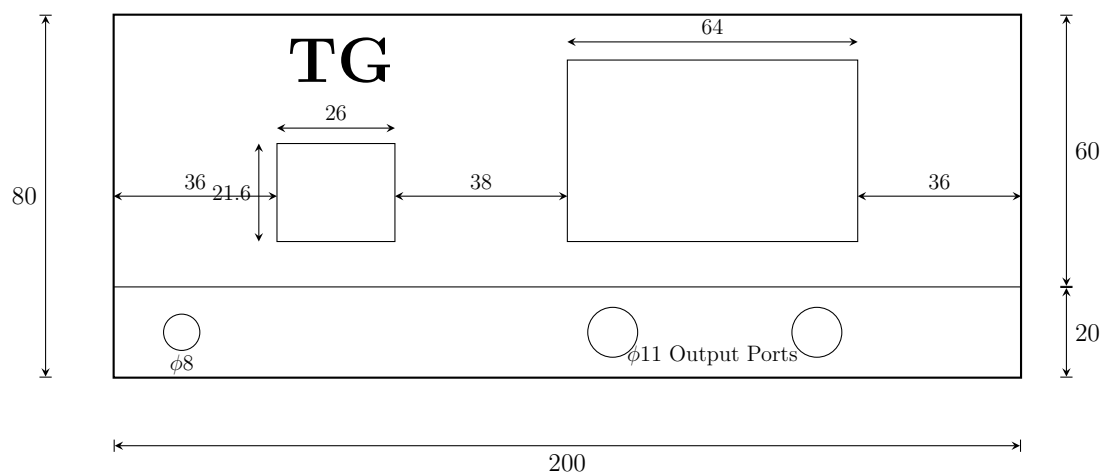


Figure 7: Schematic Diagram of Function Generator Design (Front View)

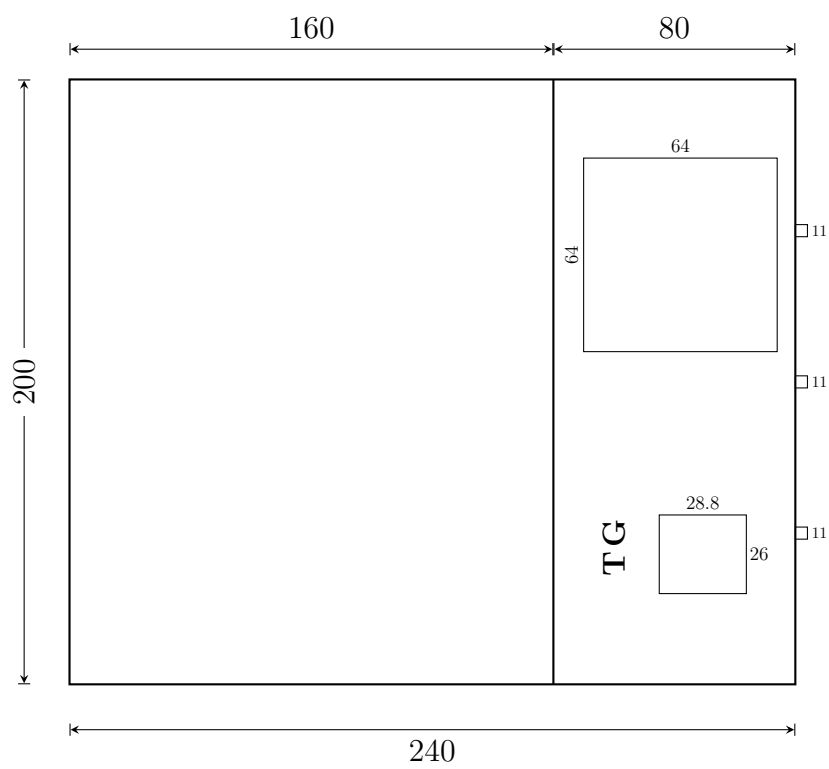


Figure 8: Schematic Diagram of Function Generator Design (Top View)

## 3.2 Core Components

### 3.2.1 ESP32 Microcontroller

- **Inputs:** Numeric keypad (20-key input interface)
- **Outputs:** OLED display control signals and SPI communication lines to the AD9833 function generator IC
- **Function:** Acts as the central control unit of the system. The ESP32 processes user inputs from the numeric keypad, manages screen updates on the OLED display, and configures waveform parameters by controlling the AD9833 and associated signal-conditioning circuitry.

### 3.2.2 AD9833 Function Generator IC

- **Input:**
  - Supply voltage: 2.3 V – 5.5 V
  - Communication protocol: SPI
  - External clock: up to 25 MHz
- **Output:**
  - Frequency range: 0 – 1 MHz (higher frequencies possible with distortion)
  - Sine wave output:  $\sim 0.65 V_{pp}$
  - Triangle wave output:  $\sim 0.65 V_{pp}$
  - Square wave output: Available (limited quality)
- **Function:**
  - Direct Digital Synthesis (DDS) based waveform generator
  - Programmable frequency and phase control
  - Generates sine, triangle, and square waves
- **Features:**
  - Low-cost and widely available
  - Stable sine and triangle outputs
  - Easy interfacing using SPI
- **Limitations:**

- Not a true Arbitrary Waveform Generator (AWG)
- Low output amplitude
- Requires external low-pass filter and amplifier

### 3.2.3 1.3-inch OLED Display Module

- **Input:** 3.3 V DC, I<sup>2</sup>C communication
- **Output:** Displays waveform type and output status
- **Function:** Real-time display of operating parameters

### 3.2.4 Operational Amplifiers

- **Components Used:** OPA810 and LM7171 (selected based on component availability)
- **Function:** These high-speed operational amplifiers are used for signal conditioning, including waveform amplification, buffering, and DC offset generation at the output stage.

## 3.3 Power Specification

### 3.3.1 Power Input

- Input voltage requirement: 12 V DC
- Maximum power consumption: 12 W
- DC input via a standard DC power supply

### 3.3.2 Output Signal Specifications

- Amplitude Range (Peak to Peak Voltage): Max peak to peak 20 V
- Load Impedance: Standard 50  $\Omega$ .

### 3.3.3 Environmental Power Considerations

- **Operating Temperature:** Usually 0 °C to 40 °C
- **Storage Temperature:** Typically -20 °C to 60 °C

## 3.4 App Innovations

### 3.4.1 Software-Defined "Safe Limits" (Protection Mode)

- In a real lab, it is very easy to accidentally blow up a sensitive component (like a low-voltage IC) by sending too much voltage.
- **The Innovation:** A user-programmable Clamp/Limit feature.
- **Software Implementation:** The user sets a Max Voltage in the App or Menu. If they (or someone else) tries to turn the physical knob past 3.3V, the software ignores the input and triggers a visual warning on the OLED/App.

### 3.4.2 Automated Frequency Sweep for Testing

- Engineers often need to find the resonant frequency of a circuit or test a filter's cutoff. Doing this manually with a knob is tedious.
- **The Innovation:** An Automated Sweep Tool.
- **Software Implementation:** The software team writes a loop that increments the frequency from a Start to Stop value over a specific Duration

### 3.4.3 Component Preset Library

- **The Idea:** Instead of just setting "1kHz Sine," the user selects "Test 741 Op-Amp" or "Test LED."
- **Software Implementation:** The app/display automatically loads the standard frequency and safe voltage levels for that specific component.

### 3.4.4 Non-Volatile Memory (NVM) Recall

- When the user turns the device off and on, it should remember exactly what frequency it was on. A real generator never resets to "0" by default.

### 3.4.5 Firmware-over-the-Air (FOTA)

- Use the WiFi capability to update the device software through the app. This is a massive selling point in modern electronics.

### 3.5 Bill of Materials

Table 2: Component Cost Table

| Component                                                 | Quantity | Cost (INR)  | Link                     |
|-----------------------------------------------------------|----------|-------------|--------------------------|
| ESP32 Node MCU Development Board                          | 1        | 275         | <a href="#">Buy Here</a> |
| OLED Display (1.3 inch I2C, 4-pin)                        | 1        | 169         | <a href="#">Buy Here</a> |
| AD9833                                                    | 2        | 293*2=586   | <a href="#">Buy Here</a> |
| Opamp (Texas Instruments OPA810IDBVR)                     | 1        | 207         | <a href="#">Buy Here</a> |
| BNC Connector Female 50 $\Omega$ Right-Angled (PCB Mount) | 2        | 59*2=118    | <a href="#">Buy Here</a> |
| ON/OFF Rocker Switch (KCD1-1-101 ON-OFF)                  | 1        | 6           | <a href="#">Buy Here</a> |
| Numpad                                                    | 1        | 71          | <a href="#">Buy Here</a> |
| Buck Converter (in starter kit)                           | 2        | 2*44=88     | <a href="#">Buy Here</a> |
| Verro Board                                               | 1        | -           | -                        |
| Female Connector (USB Type-C 4-Pin)                       | 1        | 29          | <a href="#">Buy Here</a> |
| Resistors (various)                                       | 1 set    | 20          | <a href="#">Buy Here</a> |
| Capacitors (various)                                      | 1 set    | 16          | <a href="#">Buy Here</a> |
| 2.54 mm Female Header                                     | 70       | -           | -                        |
| Acrylic Sheets (2 mm)                                     | 1        | 160         | <a href="#">Buy Here</a> |
| <b>Total Cost</b>                                         |          | <b>1745</b> |                          |

## 4 Design

This section describes the design process followed for developing the first prototype of the function generator. The aim of this stage is to verify the feasibility of the architecture and test the integration of hardware and software components.

The design process involves mechanical layout planning, electronic circuit implementation, firmware development, and preliminary testing. Since this work focuses on a prototype, the design is iterative and may undergo multiple revisions during development.

### 4.1 CAD Design

Computer-Aided Design (CAD) tools were used to visualize the physical structure of the prototype and determine approximate enclosure dimensions. The CAD model helped estimate the placement of important components such as the display, connectors, and control elements.

#### **Design considerations included**

- Overall device dimensions
- Placement of front panel components
- Space required for internal electronics
- Accessibility of controls for the user

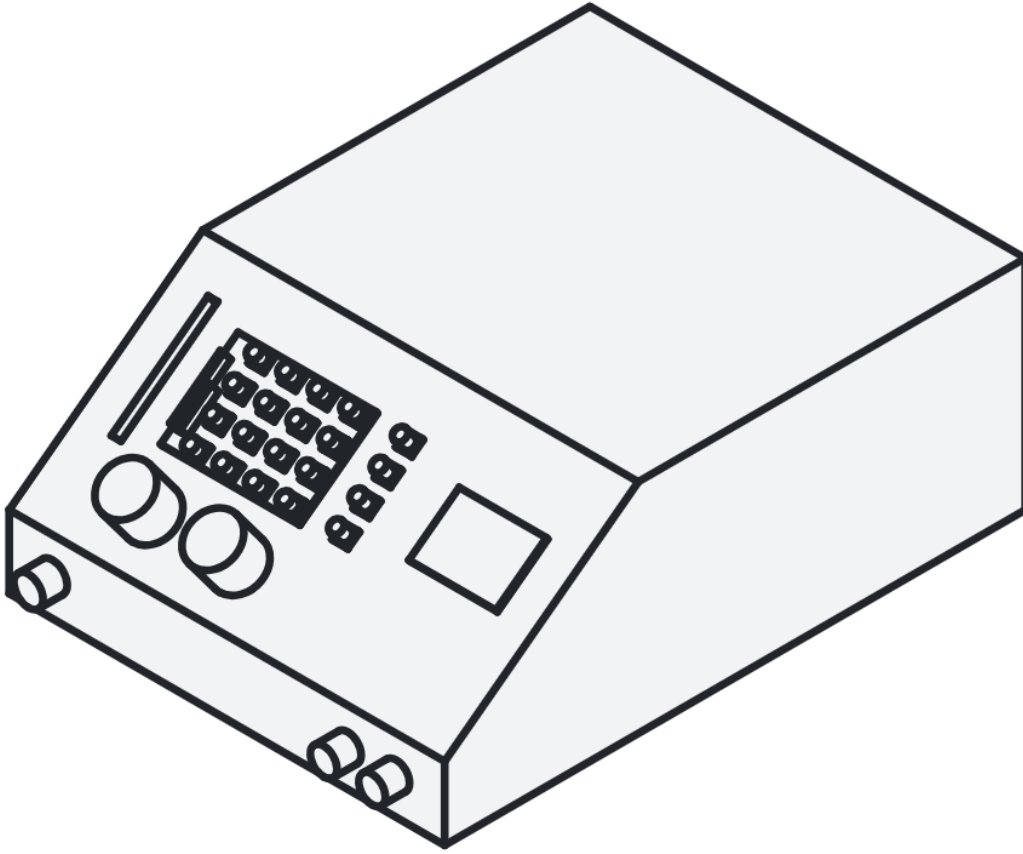


Figure 9: CAD model of the prototype enclosure

## 4.2 Prototype Implementation

After finalizing the CAD design, a physical prototype of the function generator enclosure was fabricated using acrylic sheets. The enclosure houses the electronic circuitry, user interface components, and display module.

The acrylic panels were cut using laser cutting based on the CAD net design. The sheets were then bent using a heat gun to achieve the required enclosure shape. Individual panels were joined using chloroform, which chemically bonds acrylic surfaces to form strong seals.

The internal electronics include the ESP32 microcontroller, AD9833 waveform generator IC, OLED display, keypad input module, and signal conditioning circuitry. These components were mounted using prototype boards and interconnected using jumper wires for testing and development. During early hardware testing, the OLED display was connected to the ESP32 on a breadboard to verify the display driver, waveform visualization, and parameter rendering before integration into the enclosure.

Figures below show the completed prototype and internal arrangement of the hardware components.

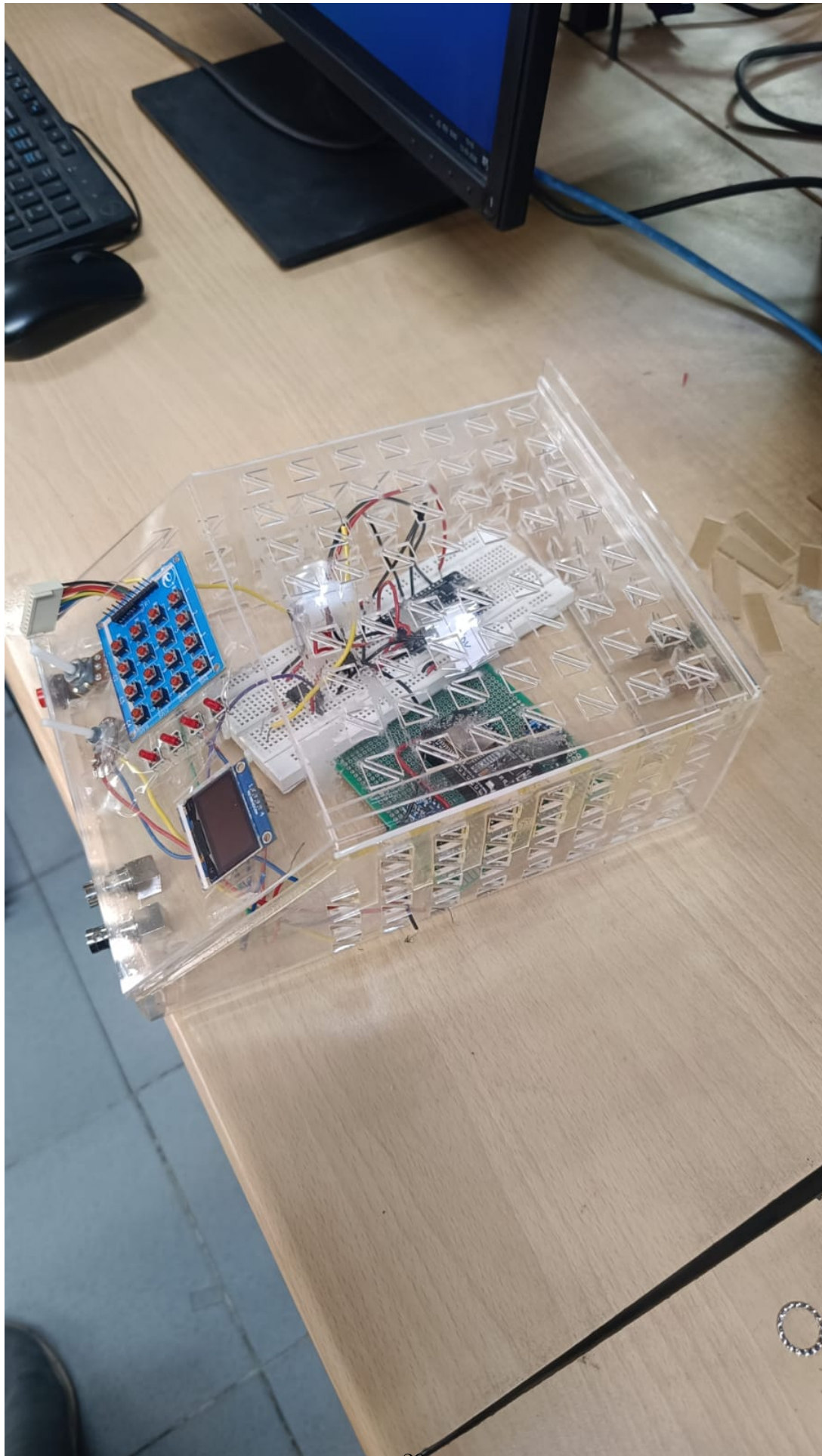


Figure 10: Internal electronics and wiring inside the prototype

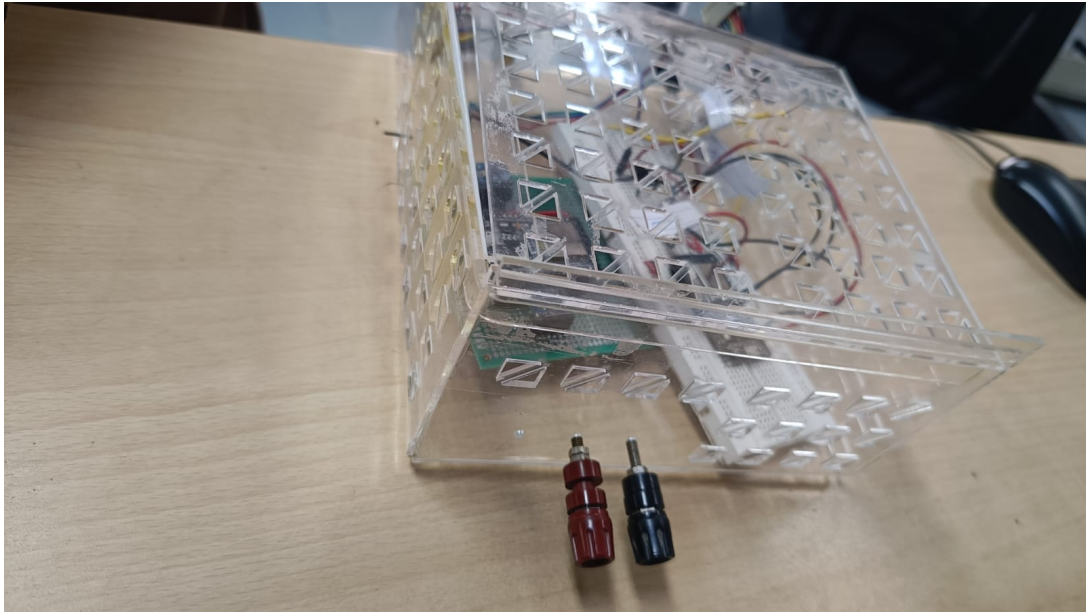


Figure 11: Prototype acrylic enclosure of the function generator

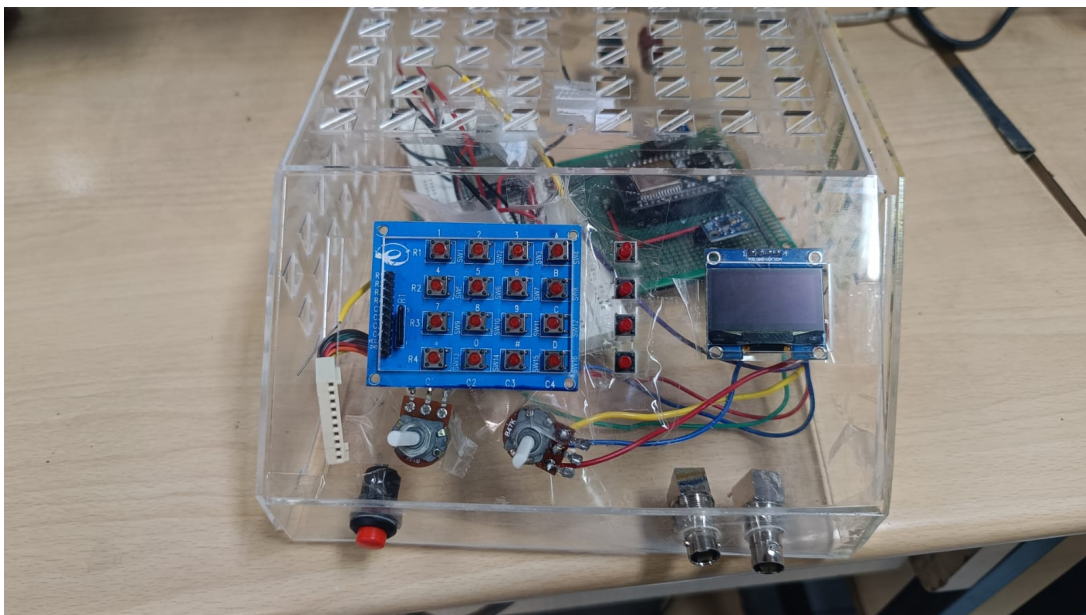


Figure 12: Front panel layout showing keypad, display and controls



Figure 13: Initial hardware testing of the OLED display module connected to the ESP32 microcontroller on a breadboard. The screen shows the generated waveform and operating parameters during early firmware validation.

### 4.3 CAD Diagrams

Different CAD views were generated to understand the structure and layout of the device.

The following diagrams will be included:

- Front panel layout
- Top view
- Side view
- Internal component layout

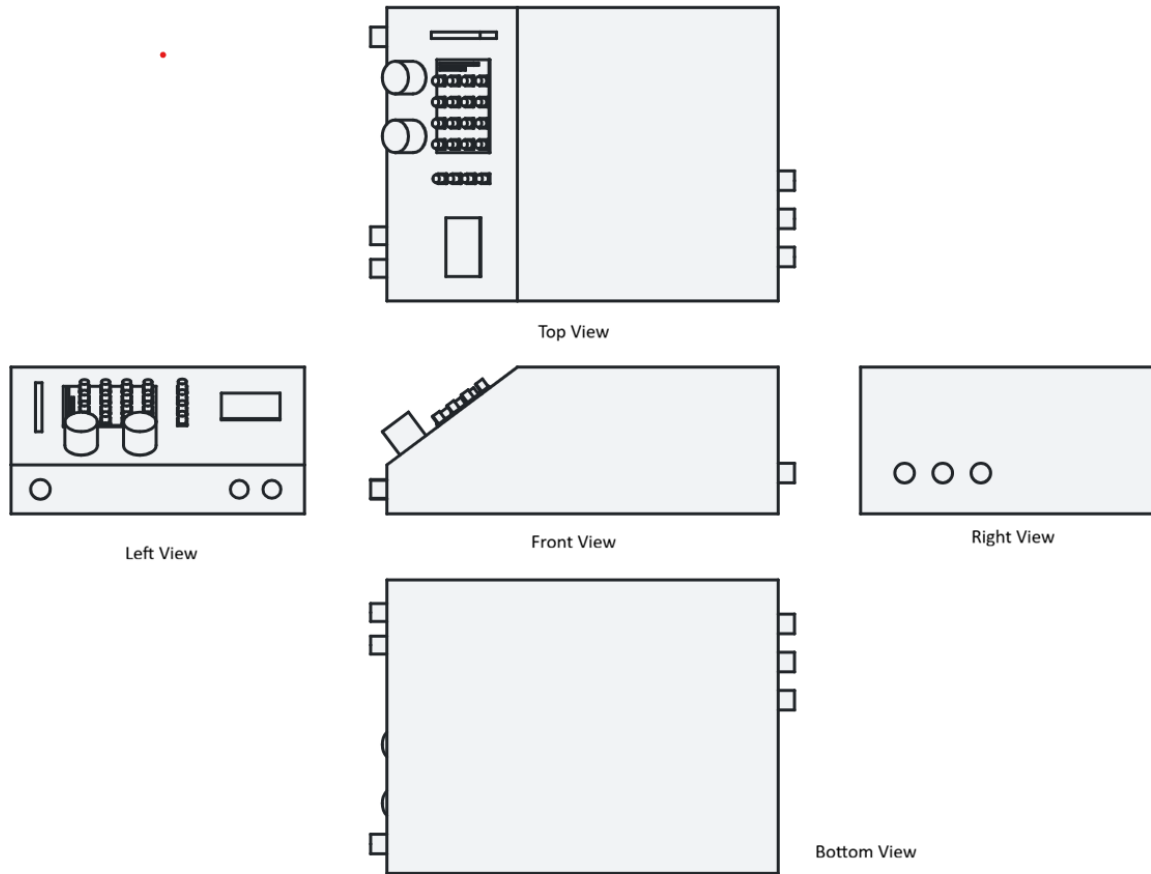


Figure 14: Front, top, bottom and side views of the function generator.

#### 4.4 Testing of CAD Design

Before fabrication, the CAD model was evaluated to ensure that the layout is practical and that components can be assembled without interference.

The following checks were considered:

- Alignment of display cutouts
- Space for connectors and switches
- Clearance for wiring and mounting
- Structural stability of the enclosure

#### 4.5 Display Interface Design

The prototype includes a display module to provide feedback to the user regarding waveform parameters and device status.

The display interface is connected to the microcontroller and updated in real time as user inputs are received.

Listing 1: Display control module

```

1
2 #ifndef DISPLAY_H
3 #define DISPLAY_H
4
5 #include <Arduino.h>
6 #include "../app/app.h"    // For AppState
7
8 // Initialize display subsystem
9 void display_init(void);
10
11
12 // Update display based on current application state
13 void display_update(const AppState* state);
14
15
16
17 enum WaveType { W_SINE=0, W_SQUARE=1, W_TRI=2 };
18
19 enum Channel { CH1=0, CH2 };
20
21 struct ScreenState {
22     double freq = 1000.00;
23     WaveType wave = W_SINE;
24     double amp = 5.00;
25     double offset = 0.00;
26     bool outOn = true;
27     int cursorPos = 0;
28     bool editing = false;
29 };
30
31
32 #endif // DISPLAY_H

```

## 4.6 Hardware Design

The hardware architecture of the prototype consists of several functional blocks including the controller, waveform generator, signal conditioning stage, and output interface.

A block diagram of the system architecture is included to show the interaction between these modules.

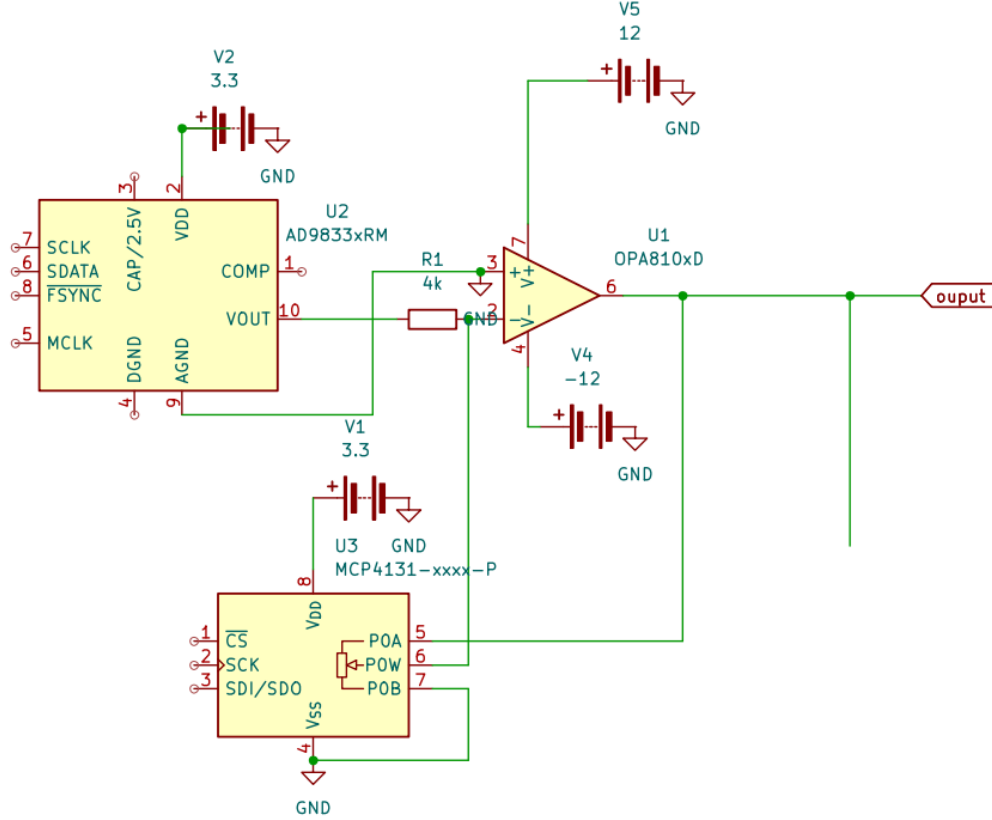


Figure 15: Circuit diagram of the waveform generation system (placeholder)

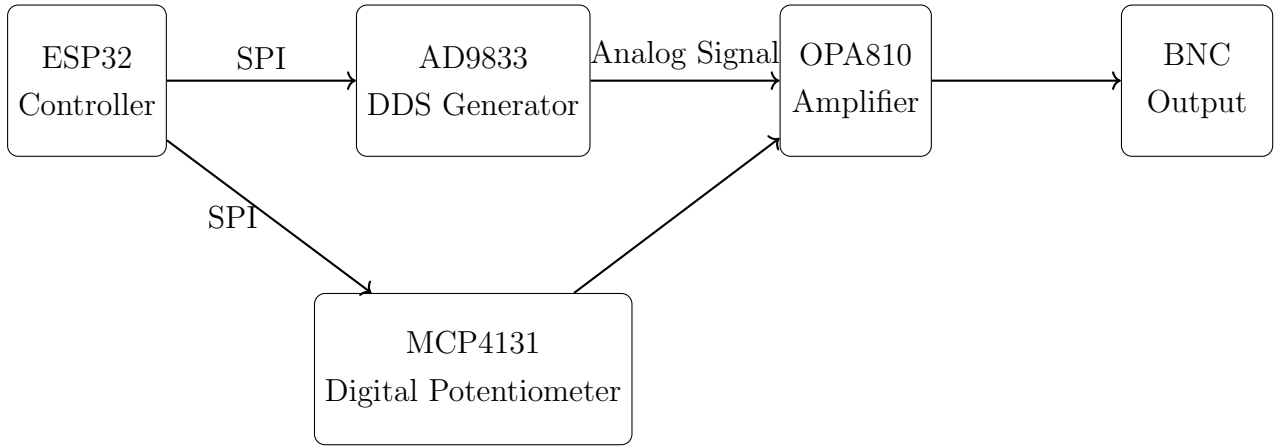


Figure 16: Block diagram of the waveform generation hardware architecture

## 4.7 Front Panel Usage

The front panel serves as the primary interface between the user and the function generator. It contains all input controls, display elements, and output connectors required for operating the device.

The layout is designed to provide clear visibility of the displayed parameters while allowing convenient access to the control elements.

The front panel includes the following components:

- **OLED Display** for displaying waveform parameters such as frequency and amplitude.
- **Numeric Keypad** for entering frequency values.
- **Additional Control Buttons** for navigation and waveform selection.
- **Potentiometers** for analog control of parameters such as amplitude or offset.
- **ON/OFF Switch** for powering the device.
- **BNC Connectors** for waveform output connections.
- **Wire Slot** for routing internal wiring to the electronics mounted inside the enclosure.

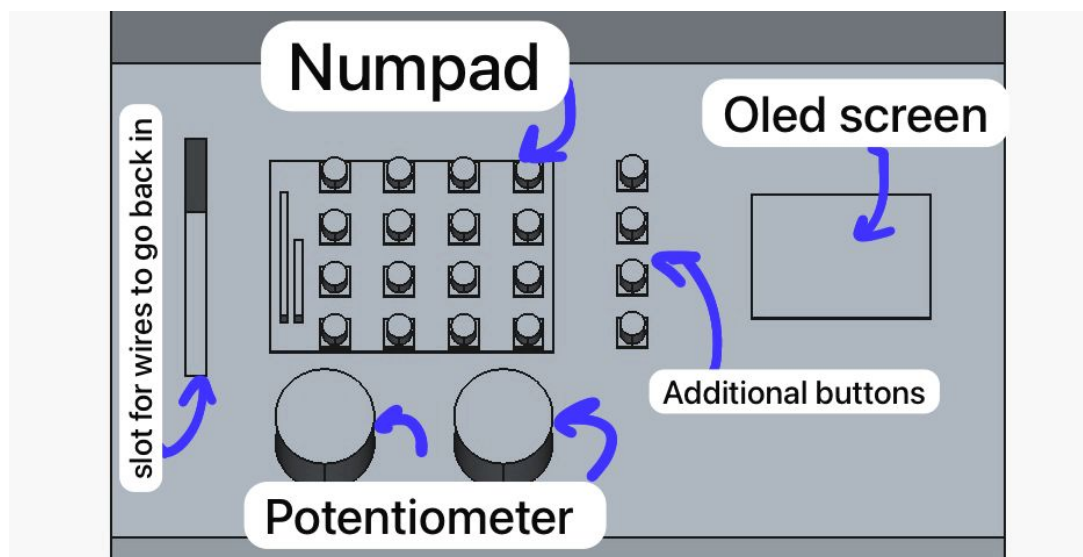


Figure 17: Annotated front panel layout showing keypad, OLED display, control buttons, potentiometers, and cable routing slot.



Figure 18: Bottom section of the front panel showing the power switch and BNC output connectors.

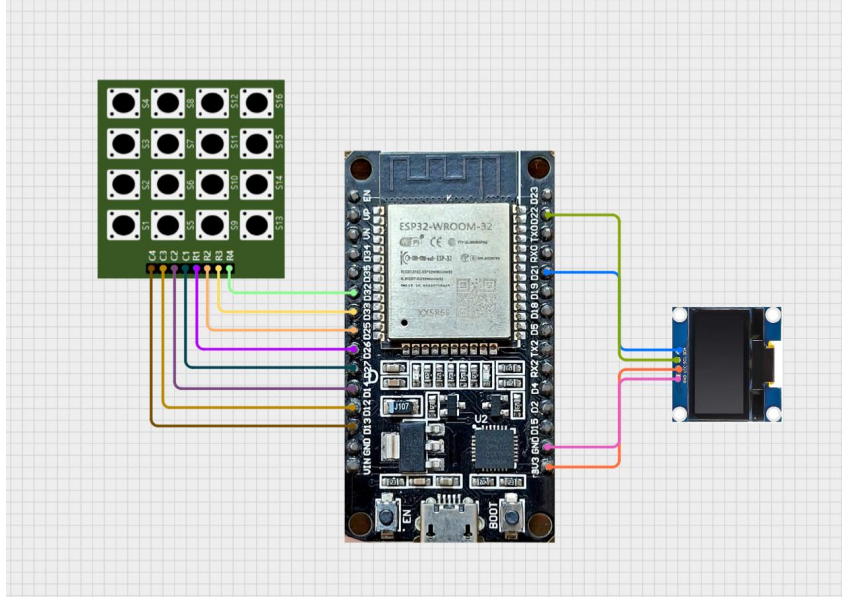


Figure 19: Electrical connections for the front panel components

## 4.8 Firmware and Embedded Code

The firmware for the function generator is implemented in C and runs on the ESP32 microcontroller. The project follows a modular architecture where different subsystems such as signal generation, display control, and input handling are separated into independent modules.

### 4.8.1 Firmware Directory Structure

The firmware is organized into multiple directories to improve maintainability and modularity.

- **app** – Main application logic and configuration
- **display** – OLED display control and GUI
- **drivers** – Hardware drivers for peripherals
- **input** – Input handling from keypad and controls
- **signal** – Waveform generation and signal processing

### 4.8.2 Application Module

Listing 2: Main application logic (app)

```

1
2 #ifndef APP_H

```

```

3  #define APP_H
4
5  // -----
6  // Includes
7  // -----
8  #include <Arduino.h>
9
10 // -----
11 // Public Types
12 // -----
13
14 // Supported waveform types
15 typedef enum {
16     WAVE_SINE = 0,
17     WAVE_SQUARE,
18     WAVE_TRIANGLE,
19     WAVEFORM_COUNT
20     // Future: SAW, NOISE, etc.
21 } WaveformType;
22
23 // Output channel selection
24 typedef enum {
25     CHANNEL_NONE = 0,
26     CHANNEL_1,
27     CHANNEL_2,
28     CHANNEL_BOTH,
29     CHANNEL_MODE_COUNT
30 } ChannelMode;
31
32 // High-level application mode
33 typedef enum {
34     APP_VIEW_MODE = 0,    // Normal operation
35     APP_EDIT_MODE        // Editing parameters
36 } AppMode;
37
38 // What parameter is currently being edited
39 // * might need changes
40 typedef enum {
41     EDIT_NONE = 0,
42     EDIT_FREQ,
43     EDIT_AMP,
44     EDIT_DC,
45     EDIT_TARGET_COUNT
46 } EditTarget;
47
48 // Magnitude for numeric input
49 typedef enum {

```

```

50     MAG_HZ = 0,
51     MAG_KHZ,
52     MAG_MHZ,
53     MAG_COUNT
54 } Magnitude;
55
56
57 typedef struct {
58     float frequency;
59     float amplitude;
60     float dc_offset;
61     WaveformType waveform;
62 } ChannelState;
63
64
65 // -----
66 // Application State
67 // -----
68
69 typedef struct {
70     ChannelState ch[2];          // ch[0] = CH1, ch[1] = CH2
71
72     ChannelMode output_mode; // NONE / CH1 / CH2 / BOTH
73     int active_channel;      // 0 or 1 (UI focus)
74
75     AppMode mode;            // view or edit mode
76     EditTarget edit_target;  // What is being edited
77
78     int digits[4];           // 4-digit input buffer
79     int digit_index;         // Active digit (0 3 )
80     Magnitude magnitude;     // Hz / kHz / MHz (or power of 10
81                               multiplier)
82     bool edit_negative;      // Sign for current edit buffer
83
84     bool output_enabled;     // * might not be needed
85 } AppState;
86
87 // -----
88 // Public API
89 // -----
90
91 // Initialize the application (called once from setup)
92 void app_init(void);
93
94 // Main application loop (called repeatedly from loop)
95 void app_run(void);

```

```

96
97 // Get current application state (read-only access)
98 const AppState* app_get_state(void);
99
100 #endif // APP_H

```

Listing 3: Application configuration (appconfig)

```

1
2 #ifndef APP_CONFIG_H
3 #define APP_CONFIG_H
4
5 // -----
6 // Default Parameters
7 // -----
8
9 #define DEFAULT_FREQUENCY_HZ      1000.0f
10 #define DEFAULT_AMPLITUDE        1.0f      // Normalized or Volts (your
    choice)
11 #define DEFAULT_DC_OFFSET        0.0f
12 #define DEFAULT_WAVEFORM         WAVE_SINE
13 #define DEFAULT_OUTPUT_STATE     false
14
15 // -----
16 // Allowed Limits
17 // -----
18
19 #define MIN_FREQUENCY_HZ          1.0f
20 #define MAX_FREQUENCY_HZ          100000.0f // Adjust based on DAC
    method
21
22 #define MIN_AMPLITUDE             0.0f
23 #define MAX_AMPLITUDE            3.3f      // ESP32 reference or op-
    amp range
24
25 #define MIN_DC_OFFSET            -1.65f
26 #define MAX_DC_OFFSET            1.65f
27
28 // -----
29 // System Timing
30 // -----
31
32 #define APP_LOOP_DELAY_MS         5          // Loop pacing (optional)
33
34 // -----
35 // Debug
36 // -----

```

```

37
38 #define ENABLE_DEBUG_LOG          1
39
40 #endif // APP_CONFIG_H

```

### 4.8.3 Display Module

Listing 4: Display control module

```

1
2 #ifndef DISPLAY_H
3 #define DISPLAY_H
4
5 #include <Arduino.h>
6 #include "../app/app.h" // For AppState
7
8 // Initialize display subsystem
9 void display_init(void);
10
11
12 // Update display based on current application state
13 void display_update(const AppState* state);
14
15
16
17 enum WaveType { W_SINE=0, W_SQUARE=1, W_TRI=2 };
18
19 enum Channel { CH1=0, CH2 };
20
21 struct ScreenState {
22     double freq = 1000.00;
23     WaveType wave = W_SINE;
24     double amp = 5.00;
25     double offset = 0.00;
26     bool outOn = true;
27     int cursorPos = 0;
28     bool editing = false;
29 };
30
31
32 #endif // DISPLAY_H

```

### 4.8.4 Hardware Drivers

Listing 5: DAC driver

```

1
2 #include <MD_AD9833.h>
3 #include <SPI.h>
4
5 // --- PIN DEFINITIONS ---
6 const uint8_t FNC_PIN = 5; // GPIO 5 (Chip Select)
7 const uint8_t LED_PIN = 2; // GPIO 2 (Onboard LED)
8
9 // Create the generator object
10 MD_AD9833 gen(FNC_PIN);
11
12 void setup() {
13     // 1. Initialize Serial
14     Serial.begin(115200);
15     while (!Serial);
16     Serial.println("\n--- ESP32 AD9833 Square Wave Generator ---");
17
18     // 2. CRITICAL FIX: Configure the LED pin as an Output
19     pinMode(LED_PIN, OUTPUT);
20
21     // 3. Initialize ESP32 Hardware SPI
22     //     SCK=18, MISO=19, MOSI=23, SS=5
23     SPI.begin(18, 19, 23, 5);
24
25     // 4. Initialize the AD9833
26     gen.begin();
27
28     // 5. Force Reset (Clear registers)
29     gen.setMode(MD_AD9833::MODE_OFF);
30     delay(10);
31
32     // 6. Set Output to SQUARE WAVE (Changed from MODE_SINE)
33     gen.setMode(MD_AD9833::MODE_SINE);
34     // gen.setMode(MD_AD9833::MODE_SQUARE1);
35     // gen.setMode(MD_AD9833::MODE_TRIANGLE);
36
37     // 7. Set Frequency to 1 kHz (1000.0 Hz)
38     gen.setFrequency(MD_AD9833::CHAN_0, 10.0);
39
40     Serial.println("Status: Generating 1 kHz Square Wave");
41 }
42
43 void loop() {
44     // 8. Blink the LED explicitly
45     digitalWrite(LED_PIN, HIGH); // LED ON
46     delay(500); // Wait 0.5 second
47

```

```

48     digitalWrite(LED_PIN, LOW);    // LED OFF
49     delay(500);                    // Wait 0.5 second
50 }

```

### 4.8.5 Input Handling

Listing 6: Input processing module

```

1
2 #pragma once
3 #include <Arduino.h>
4
5 // Events understood by app.cpp ONLY
6 typedef enum
7 {
8     UI_NONE = 0,
9
10    // digits
11    UI_NUM_0,
12    UI_NUM_1,
13    UI_NUM_2,
14    UI_NUM_3,
15    UI_NUM_4,
16    UI_NUM_5,
17    UI_NUM_6,
18    UI_NUM_7,
19    UI_NUM_8,
20    UI_NUM_9,
21
22    // edit control
23    UI_CONFIRM,          // 'Enter' button in EDIT mode
24    // view mode toggles
25    UI_WAVEFORM_TOGGLES,
26    UI_ACTIVE_CHANNEL_TOGGLES,
27    UI_OUTPUT_CHANNEL_TOGGLES,
28    UI_MODIFY_TOGGLES,   // Enter edit mode
29
30    // edit helper
31    UI_MAGNITUDE_TOGGLES,
32    UI_NEXT_FIELD,
33
34 } UIEvent;
35
36
37 // API
38 void input_init(void);
39 void input_update(void);

```

```

40 UIEvent input_get_event(void);
41 // push external events (web / serial / bluetooth etc)
42 void input_inject_event(UIEvent e);

```

Listing 7: Input mapping module

```

1
2 #pragma once
3 #include "input.h"
4 #include <Arduino.h>
5
6 // ----- Keypad size -----
7 #define INPUT_ROWS 4
8 #define INPUT_COLS 4
9
10 // ----- Row pins -----
11 static byte INPUT_ROW_PINS[INPUT_ROWS] = {32, 33, 25, 26};
12
13 // ----- Column pins -----
14 static byte INPUT_COL_PINS[INPUT_COLS] = {27, 14, 12, 13};
15
16 // ----- Key layout (characters returned by keypad library) -----
17 static char KEYPAD_KEYS[INPUT_ROWS][INPUT_COLS] =
18 {
19     {'1','2','3','A'},
20     {'4','5','6','C'},
21     {'7','8','9','B'},
22     {'*','0','#','D'}
23 };

```

#### 4.8.6 Signal Generation

Listing 8: Signal generator implementation

```

1
2 #ifndef SIGNAL_GEN_H
3 #define SIGNAL_GEN_H
4
5 #include <Arduino.h>
6 #include "../app/app.h"
7
8 // Initialize signal generator hardware
9 void signal_init(void);
10
11 // Per-channel setters
12 void signal_set_frequency(int ch, float freq);
13 void signal_set_amplitude(int ch, float amp);

```

```

14 void signal_set_offset(int ch, float offset);
15 void signal_set_waveform(int ch, WaveformType w);
16
17 // Output routing
18 void signal_set_output_mode(ChannelMode mode);
19
20 #endif

```

## 4.9 Web-Based Control Interface

In addition to the physical control interface, the system also provides a web-based interface that allows the function generator to be controlled remotely using a browser. The ESP32 microcontroller hosts a lightweight web server which serves a user interface to connected devices over Wi-Fi.

This interface allows the user to monitor parameters and control the function generator without requiring a dedicated mobile application.

### 4.9.1 Web Application Architecture

The web control system consists of two main components:

- **Frontend Interface (index.html)** – A browser-based user interface that allows users to interact with the device.
- **Backend Firmware (sketch.ino)** – The ESP32 firmware that hosts the web server and processes requests from the web interface.

### 4.9.2 Web Interface Implementation

Listing 9: Web application user interface (index.html)

```

1
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8" />
6   <meta name="viewport" content="width=device-width, initial-scale=1.0"
7     />
8   <title>Dual Channel Function Generator</title>
9   <link href="https://fonts.googleapis.com/css2?family=Space+Mono:
    wght@400;700&family=Syne:wght@400;600;700;800&display=swap" rel="
    stylesheet"/>
10  <style>

```

```

10  /*

11      TOKENS & RESET
12
13      */
14
15  *, *::before, *::after { box-sizing: border-box; margin: 0; padding
16      : 0; }
17
18  :root {
19      --radius-sm: 6px;
20      --radius-md: 10px;
21      --radius-lg: 16px;
22      --radius-xl: 22px;
23
24      --bg: #080b10;
25      --surface: #0e1319;
26      --surface-2: #131920;
27      --border: #1c2330;
28      --border-light: #2a3545;
29
30      --ch1: #00ffaa;
31      --ch1-dim: #00ffaa22;
32      --ch1-glow: #00ffaa55;
33      --ch2: #ff6b35;
34      --ch2-dim: #ff6b3522;
35      --ch2-glow: #ff6b3555;
36
37      --text-primary: #dde8f5;
38      --text-sec: #6b7f99;
39      --text-muted: #384556;
40      --font-ui: 'Syne', sans-serif;
41      --font-mono: 'Space Mono', monospace;
42
43      --active: var(--ch1);
44      --active-dim: var(--ch1-dim);
45      --active-glow: var(--ch1-glow);
46  }
47
48  body {
49      background: var(--bg);
50      font-family: var(--font-ui);
51      min-height: 100vh;
52      display: flex;
53      justify-content: center;
54      align-items: flex-start;
55      padding: 28px 16px 60px;

```

```

53     color: var(--text-primary);
54     background-image:
55         linear-gradient(rgba(0,255,170,.025) 1px, transparent 1px),
56         linear-gradient(90deg, rgba(0,255,170,.025) 1px, transparent 1
           px);
57     background-size: 40px 40px;
58 }
59
60 .app { width: 100%; max-width: 500px; }
61
62 /*
63
64     HEADER
65
66     */
67 .header {
68     display: flex;
69     align-items: center;
70     gap: 16px;
71     padding: 20px 22px;
72     background: var(--surface);
73     border: 1px solid var(--border);
74     border-radius: var(--radius-xl);
75     margin-bottom: 14px;
76     position: relative;
77     overflow: hidden;
78 }
79 .header::before {
80     content: '';
81     position: absolute;
82     inset: 0;
83     background: linear-gradient(135deg, var(--ch1-dim) 0%,
84         transparent 60%);
85     pointer-events: none;
86 }
87 .header-icon {
88     width: 46px; height: 46px;
89     border-radius: var(--radius-md);
90     background: var(--ch1);
91     display: flex; align-items: center; justify-content: center;
92     font-size: 22px;
93     box-shadow: 0 0 24px var(--ch1-glow);
94     flex-shrink: 0;
95     position: relative;
96 }

```

```

94 .header-title { font-size: 16px; font-weight: 800; letter-spacing:
    0.02em; text-transform: uppercase; }
95 .header-sub { font-family: var(--font-mono); font-size: 11px; color
    : var(--text-sec); margin-top: 3px; letter-spacing: 0.08em; }
96 .header-badge {
97     margin-left: auto; font-family: var(--font-mono); font-size: 10px
    ; padding: 4px 10px;
98     border-radius: 999px; background: var(--ch1-dim); color: var(--
    ch1);
99     border: 1px solid var(--ch1-glow); white-space: nowrap; flex-
    shrink: 0;
100 }
101
102 /*
103
104     CHANNEL TABS
105
106     */
105 .tab-bar { display: grid; grid-template-columns: 1fr 1fr; gap: 8px;
    margin-bottom: 14px; }
106 .tab-btn {
107     padding: 13px 0; border-radius: var(--radius-md); border: 1px
    solid var(--border-light);
108     background: var(--surface); color: var(--text-sec); font-weight:
    700; font-size: 13px;
109     cursor: pointer; font-family: var(--font-ui); text-transform:
    uppercase; letter-spacing: 0.06em;
110     transition: all 0.2s ease; display: flex; align-items: center;
    justify-content: center; gap: 9px;
111 }
112 .tab-btn:hover:not(.active-ch1):not(.active-ch2) { border-color:
    var(--border-light); color: var(--text-primary); }
113 .tab-dot { width: 8px; height: 8px; border-radius: 50%; }
114 .tab-btn.active-ch1 {
115     background: var(--ch1-dim); color: var(--ch1); border-color: var
    (--ch1-glow);
116     box-shadow: inset 0 0 20px var(--ch1-dim), 0 0 12px var(--ch1-dim
    );
117 }
118 .tab-btn.active-ch2 {
119     background: var(--ch2-dim); color: var(--ch2); border-color: var
    (--ch2-glow);
120     box-shadow: inset 0 0 20px var(--ch2-dim), 0 0 12px var(--ch2-dim
    );
121 }
122

```

```

123  /*
124
125      CHANNEL PANEL
126
127      */
128  .ch-panel { background: var(--surface); border: 1px solid var(--
129      border); border-radius: var(--radius-xl); overflow: hidden; }
130  .ch-panel-header { display: flex; align-items: center; justify-
131      content: space-between; padding: 14px 20px; border-bottom: 1px
132      solid var(--border); background: var(--surface-2); }
133  .ch-panel-label { display: flex; align-items: center; gap: 10px; }
134  .ch-dot { width: 10px; height: 10px; border-radius: 50%; flex-
135      shrink: 0; }
136  .ch-name { font-size: 13px; font-weight: 700; letter-spacing: 0.08
137      em; text-transform: uppercase; }
138  .ch-wave-badge {
139      font-family: var(--font-mono); font-size: 10px; padding: 3px 9px;
140      border-radius: 999px;
141      background: var(--active-dim); color: var(--active); border: 1px
142      solid var(--active-glow); text-transform: uppercase; letter-
143      spacing: 0.08em;
144  }
145
146  /* Oscilloscope */
147  .canvas-wrap { padding: 14px 16px; background: var(--bg); border-
148      bottom: 1px solid var(--border); position: relative; }
149  .canvas-corner { position: absolute; font-family: var(--font-mono);
150      font-size: 9px; color: var(--text-muted); letter-spacing: 0.06
151      em; }
152  .canvas-corner.tl { top: 20px; left: 22px; } .canvas-corner.tr {
153      top: 20px; right: 22px; }
154  .canvas-corner.bl { bottom: 20px; left: 22px; } .canvas-corner.br {
155      bottom: 20px; right: 22px; }
156  canvas { width: 100%; height: 170px; border-radius: var(--radius-md
157      ); display: block; }
158
159  /* Stats */
160  .stats-grid { display: grid; grid-template-columns: 1fr 1fr; gap:
161      0; border-bottom: 1px solid var(--border); }
162  .stat-cell { padding: 16px 20px; position: relative; }
163  .stat-cell:first-child { border-right: 1px solid var(--border); }
164  .stat-label { font-family: var(--font-mono); font-size: 10px; color
165      : var(--text-sec); letter-spacing: 0.1em; text-transform:
166      uppercase; margin-bottom: 6px; }
167  .stat-value { display: flex; align-items: baseline; font-family:
168      var(--font-mono); font-size: 28px; font-weight: 700; letter-

```

```

    spacing: -0.03em; line-height: 1; }
149 .stat-input { background: transparent; border: none; color: inherit
    ; font-family: inherit; font-size: inherit; font-weight: inherit
    ; letter-spacing: inherit; width: 3.5em; padding: 0; margin: 0;
    outline: none; }
150 .stat-input::-webkit-outer-spin-button, .stat-input::-webkit-inner-
    spin-button { -webkit-appearance: none; margin: 0; }
151 .stat-input[type=number] { -moz-appearance: textfield; appearance:
    textfield; }
152 .stat-select { background: transparent; border: none; color:
    inherit; font-family: inherit; outline: none; cursor: pointer;
    padding: 0; -webkit-appearance: none; -moz-appearance: none;
    appearance: none; }
153 .stat-select option { background: var(--surface-2); color: var(--
    text-primary); }
154 .stat-unit { font-size: 12px; font-weight: 400; margin-left: 2px;
    opacity: 0.7; }
155
156 /* Controls */
157 .controls-section { padding: 18px 20px; display: flex; flex-
    direction: column; gap: 18px; }
158 .section-label { font-family: var(--font-mono); font-size: 10px;
    letter-spacing: 0.12em; text-transform: uppercase; color: var(--
    text-sec); margin-bottom: 10px; }
159
160 /* Wave Buttons */
161 .wave-btns { display: grid; grid-template-columns: repeat(3, 1fr);
    gap: 8px; }
162 .wave-btn {
163     padding: 12px 6px 10px; border-radius: var(--radius-md); border:
    1px solid var(--border-light);
164     background: var(--bg); color: var(--text-sec); font-family: var
    (--font-ui); font-size: 12px;
165     font-weight: 600; cursor: pointer; transition: all 0.18s ease;
    display: flex; flex-direction: column; align-items: center;
    gap: 7px;
166 }
167 .wave-btn:hover:not(.active-ch1):not(.active-ch2) { border-color:
    var(--border-light); color: var(--text-primary); }
168 .wave-btn svg { width: 36px; height: 20px; stroke-width: 1.8;
    stroke-linecap: round; stroke-linejoin: round; fill: none; }
169 .wave-btn.active-ch1 { background: var(--ch1-dim); color: var(--ch1
    ); border-color: var(--ch1-glow); box-shadow: 0 0 12px var(--ch1
    -dim); }
170 .wave-btn.active-ch1 svg { stroke: var(--ch1); }
171 .wave-btn.active-ch2 { background: var(--ch2-dim); color: var(--ch2
    ); border-color: var(--ch2-glow); box-shadow: 0 0 12px var(--ch2

```

```

    -dim); }
172 .wave-btn.active-ch2 svg { stroke: var(--ch2); }
173 .wave-btn:not(.active-ch1):not(.active-ch2) svg { stroke: var(--
    text-muted); }
174
175 /* Sliders */
176 .slider-header { display: flex; justify-content: space-between;
    align-items: baseline; margin-bottom: 10px; }
177 .slider-current { font-family: var(--font-mono); font-size: 12px;
    color: var(--active); font-weight: 700; }
178 .slider-track-wrap { position: relative; height: 28px; display:
    flex; align-items: center; }
179
180 input[type=range] { -webkit-appearance: none; appearance: none;
    width: 100%; height: 4px; border-radius: 2px; cursor: pointer;
    outline: none; border: none; display: block; position: relative;
    z-index: 1; }
181 .slider-green { background: linear-gradient(to right, var(--ch1)
    var(--pct,0%), var(--border-light) var(--pct,0%)); }
182 .slider-green::-webkit-slider-runnable-track { height: 4px; border-
    radius: 2px; }
183 .slider-green::-webkit-slider-thumb {
184     -webkit-appearance: none; width: 20px; height: 20px; border-
    radius: 50%; background: var(--bg);
185     border: 2px solid var(--ch1); box-shadow: 0 0 14px var(--ch1-glow
    ); cursor: pointer; margin-top: -8px; transition: box-shadow
    0.15s;
186 }
187 .slider-green:active::-webkit-slider-thumb { box-shadow: 0 0 22px
    var(--ch1); }
188
189 .slider-orange { background: linear-gradient(to right, var(--ch2)
    var(--pct,0%), var(--border-light) var(--pct,0%)); }
190 .slider-orange::-webkit-slider-runnable-track { height: 4px; border
    -radius: 2px; }
191 .slider-orange::-webkit-slider-thumb {
192     -webkit-appearance: none; width: 20px; height: 20px; border-
    radius: 50%; background: var(--bg);
193     border: 2px solid var(--ch2); box-shadow: 0 0 14px var(--ch2-glow
    ); cursor: pointer; margin-top: -8px; transition: box-shadow
    0.15s;
194 }
195 .slider-orange:active::-webkit-slider-thumb { box-shadow: 0 0 22px
    var(--ch2); }
196
197 .slider-range-labels { display: flex; justify-content: space-
    between; margin-top: 6px; }

```

```

198     .slider-min, .slider-max { font-family: var(--font-mono); font-size
      : 9px; color: var(--text-muted); letter-spacing: 0.04em; }
199
200     /* Summary Bar */
201     .summary-bar { display: grid; grid-template-columns: 1fr 1fr; gap:
      10px; margin-top: 14px; }
202     .summary-chip { background: var(--surface); border: 1px solid var
      (--border); border-radius: var(--radius-md); padding: 10px 14px;
      display: flex; align-items: center; gap: 10px; }
203     .summary-pip { width: 7px; height: 7px; border-radius: 50%; flex-
      shrink: 0; }
204     .summary-ch-label { font-size: 10px; font-weight: 700; letter-
      spacing: 0.1em; text-transform: uppercase; color: var(--text-sec
      ); margin-bottom: 3px; }
205     .summary-vals { font-family: var(--font-mono); font-size: 11px;
      color: var(--text-primary); }
206     </style>
207 </head>
208 <body>
209 <div class="app">
210
211     <div class="header" id="appHeader">
212         <div class="header-icon"> </div>
213         <div>
214             <div class="header-title">Function Generator</div>
215             <div class="header-sub">DUAL CHANNEL WAVEFORM CONTROL</div>
216         </div>
217         <div class="header-badge" id="liveBadge"> LIVE</div>
218     </div>
219
220     <div class="tab-bar">
221         <button class="tab-btn active-ch1" id="tab1" onclick="UI.
      switchChannel(1)">
222             <span class="tab-dot" id="dot1" style="background: var(--ch1); box-
      shadow: 0 0 6px var(--ch1)"></span>
223             Channel 1
224         </button>
225         <button class="tab-btn" id="tab2" onclick="UI.switchChannel(2)">
226             <span class="tab-dot" id="dot2" style="background: var(--ch2); box-
      shadow: 0 0 6px var(--ch2)"></span>
227             Channel 2
228         </button>
229     </div>
230
231     <div class="ch-panel">
232         <div class="ch-panel-header">
233             <div class="ch-panel-label">

```

```

234     <div class="ch-dot" id="chDot" style="background:var(--ch1);box
        -shadow:0 0 8px var(--ch1)"></div>
235     <span class="ch-name" id="chName">Channel 1</span>
236 </div>
237     <div class="ch-wave-badge" id="chWaveBadge">Sine Wave</div>
238 </div>
239
240 <div class="canvas-wrap">
241     <div class="canvas-corner_tl" id="cornerTL">CH1</div>
242     <div class="canvas-corner_tr" id="cornerTR">SINE</div>
243     <div class="canvas-corner_bl">OSC</div>
244     <div class="canvas-corner_br" id="cornerBR">LIVE</div>
245     <canvas id="waveCanvas"></canvas>
246 </div>
247
248 <div class="stats-grid">
249     <div class="stat-cell">
250         <div class="stat-label">Frequency</div>
251         <div class="stat-value" id="freqDisplayWrap" style="color:var
            (--ch1)">
252             <input type="number" id="freqInput" class="stat-input" value=
                "10.00" step="any" onchange="UI.onFreqInput(this.value)" /
                >
253             <select class="stat-unit stat-select" id="freqUnit" onchange=
                "UI.onFreqUnitChange(this.value)">
254                 <option value="Hz">Hz</option>
255                 <option value="kHz">kHz</option>
256                 <option value="MHz">MHz</option>
257             </select>
258         </div>
259     </div>
260     <div class="stat-cell">
261         <div class="stat-label">Amplitude</div>
262         <div class="stat-value" id="ampDisplayWrap" style="color:var(--
            ch1)">
263             <input type="number" id="ampInput" class="stat-input" value="
                5.0" step="any" onchange="UI.onAmpInput(this.value)" />
264             <span class="stat-unit" id="ampUnit">V</span>
265         </div>
266     </div>
267 </div>
268
269 <div class="controls-section">
270     <div>
271         <div class="section-label">Waveform</div>
272         <div class="wave-btns">

```

```

273     <button class="wave-btn active-ch1" id="btn-sine" onclick="UI
        .setWave('sine')">
274         <svg viewBox="0 0 36 20"><path d="M2 10 C6 2, 10 2, 13 10
            C16 18, 20 18, 23 10 C26 2, 30 2, 34 10"/></svg> Sine
275     </button>
276     <button class="wave-btn" id="btn-square" onclick="UI.setWave
        ('square')">
277         <svg viewBox="0 0 36 20"><path d="M2 15 L2 5 L12 5 L12 15
            L22 15 L22 5 L32 5 L32 15 L34 15"/></svg> Square
278     </button>
279     <button class="wave-btn" id="btn-triangular" onclick="UI.
        setWave('triangular')">
280         <svg viewBox="0 0 36 20"><path d="M2 15 L9 5 L18 15 L27 5
            L34 15"/></svg> Triangle
281     </button>
282 </div>
283 </div>
284
285 <div class="slider-block">
286     <div class="slider-header">
287         <div class="section-label" style="margin-bottom:0">Frequency<
            /div>
288         <div class="slider-current" id="freqSliderVal">10.00 kHz</div>
289     </div>
290     <div class="slider-track-wrap">
291         <input type="range" class="slider-green" id="freqSlider" min=
            "10" max="1000000" value="10000" oninput="UI.onFreqChange(
                this.value)" />
292     </div>
293     <div class="slider-range-labels">
294         <span class="slider-min">10 Hz</span><span class="slider-max"
            id="freqSliderMax">1.00 MHz</span>
295     </div>
296 </div>
297
298 <div class="slider-block">
299     <div class="slider-header">
300         <div class="section-label" style="margin-bottom:0">Amplitude<
            /div>
301         <div class="slider-current" id="ampSliderVal">5.0 V</div>
302     </div>
303     <div class="slider-track-wrap">
304         <input type="range" class="slider-green" id="ampSlider" min="
            0" max="1000" value="500" oninput="UI.onAmpChange(this.
                value)" />
305 </div>

```

```

306     <div class="slider-range-labels">
307         <span class="slider-min">0 V</span><span class="slider-max"
            id="ampSliderMax">10.0 V</span>
308     </div>
309 </div>
310 </div>
311 </div>
312
313 <div class="summary-bar">
314     <div class="summary-chip">
315         <div class="summary-pip" style="background:var(--ch1);box-shadow
            :0 0 6px var(--ch1)"></div>
316         <div class="summary-info">
317             <div class="summary-ch-label">CH1</div>
318             <div class="summary-vals" id="summaryCh1">10.00 kHz      5.0 V</
                div>
319         </div>
320     </div>
321     <div class="summary-chip">
322         <div class="summary-pip" style="background:var(--ch2);box-shadow
            :0 0 6px var(--ch2)"></div>
323         <div class="summary-info">
324             <div class="summary-ch-label">CH2</div>
325             <div class="summary-vals" id="summaryCh2">1.00 kHz      3.5 V</
                div>
326         </div>
327     </div>
328 </div>
329
330 </div>
331
332 <script>
333 /*
334
335     MODULE: Network (NEW)
336     Handles API communication to the ESP32 backend
337
338     */
339 const Network = (() => {
340     function send(endpoint, params = {}) {
341         const query = new URLSearchParams(params).toString();
342         const url = `/${endpoint}${query ? '?' + query : ''}`;
343         fetch(url).catch(err => console.error('Network error:', err));
344     }
345 }

```

```

344 // Prevents flooding the ESP32 while the user is actively dragging a
      slider
345 let debounceTimer;
346 function sendDebounced(endpoint, params = {}, delay = 100) {
347     clearTimeout(debounceTimer);
348     debounceTimer = setTimeout(() => send(endpoint, params), delay);
349 }
350
351 async function getStatus() {
352     try {
353         const response = await fetch('/getStatus');
354         return await response.json();
355     } catch (err) {
356         console.error('Sync error:', err);
357         return null;
358     }
359 }
360
361 return { send, sendDebounced, getStatus };
362 })();
363
364 /*
365     MODULE: State
366
367     */
368 const State = (() => {
369     const AMP_STEPS = 1000;
370     const data = {
371         activeChannel: 1,
372         maxFreq: 1000000,
373         maxAmp: 10,
374         channels: {
375             1: { frequency: 10000, amplitude: 5, waveType: 'sine', color:
                 '#00ffaa', name: 'CH1' },
376             2: { frequency: 1000, amplitude: 3.5, waveType: 'sine', color:
                 '#ff6b35', name: 'CH2' },
377         }
378     };
379
380     return {
381         AMP_STEPS, get: () => data, getChannel: (ch) => data.channels[ch],
382         getActive: () => data.channels[data.activeChannel],
383         getActiveId: () => data.activeChannel,
384         setActiveId: (ch) => { data.activeChannel = ch; },

```

```

384     setMaxFreq: (v) => { data.maxFreq = v; }, setMaxAmp: (v) => { data.
        maxAmp = v; },
385     setFreq: (ch, v) => { data.channels[ch].frequency = v; },
386     setAmp: (ch, v) => { data.channels[ch].amplitude = v; },
387     setWave: (ch, t) => { data.channels[ch].waveType = t; },
388 };
389 })();
390
391 /*
392
393     MODULE: Format
394
395     */
396 const Format = (() => {
397     function freq(hz) {
398         if (hz >= 1e6) return `${(hz / 1e6).toFixed(2)} MHz`;
399         if (hz >= 1e3) return `${(hz / 1e3).toFixed(2)} kHz`;
400         return `${hz} Hz`;
401     }
402     function amp(v) { return `${v.toFixed(1)} V`; }
403     function freqLabel(hz) {
404         if (hz >= 1e6) return { num: (hz / 1e6).toFixed(2), unit: 'MHz' };
405         if (hz >= 1e3) return { num: (hz / 1e3).toFixed(2), unit: 'kHz' };
406         return { num: `${hz}`, unit: 'Hz' };
407     }
408     function ampLabel(v) { return { num: v.toFixed(1), unit: 'V' }; }
409     return { freq, amp, freqLabel, ampLabel };
410 })();
411
412 /*
413
414     MODULE: Oscilloscope
415
416     */
417 const Oscilloscope = (() => {
418     const canvas = document.getElementById('waveCanvas');
419     const ctx = canvas.getContext('2d');
420     let phase = 0, animId = null;
421
422     function resize() {
423         canvas.width = canvas.offsetWidth * window.devicePixelRatio ||
            canvas.offsetWidth;
424         canvas.height = canvas.offsetHeight * window.devicePixelRatio ||
            canvas.offsetHeight;
425         ctx.scale(window.devicePixelRatio, window.devicePixelRatio);

```

```

422 }
423
424 new ResizeObserver(resize).observe(canvas);
425 resize();
426
427 function drawGrid(W, H) {
428     ctx.strokeStyle = 'rgba(255,255,255,0.04)'; ctx.lineWidth = 1;
429     const cols = 8, rows = 4;
430     for (let i = 1; i < cols; i++) { ctx.beginPath(); ctx.moveTo((W/
        cols)*i, 0); ctx.lineTo((W/cols)*i, H); ctx.stroke(); }
431     for (let i = 1; i < rows; i++) { ctx.beginPath(); ctx.moveTo(0, (H/
        rows)*i); ctx.lineTo(W, (H/rows)*i); ctx.stroke(); }
432     ctx.strokeStyle = 'rgba(255,255,255,0.09)'; ctx.setLineDash([4, 6])
        ;
433     ctx.beginPath(); ctx.moveTo(0, H/2); ctx.lineTo(W, H/2); ctx.stroke
        (); ctx.setLineDash([]);
434 }
435
436 function computeY(waveType, t) {
437     if (waveType === 'sine') return Math.sin(t);
438     if (waveType === 'square') return Math.sin(t) >= 0 ? 1 : -1;
439     const pos = ((t / (2 * Math.PI)) % 1 + 1) % 1;
440     return pos < 0.5 ? 4 * pos - 1 : -4 * pos + 3;
441 }
442
443 function drawWaveform(W, H, ch) {
444     const s = State.get();
445     const cycles = Math.max(1, Math.min(12, (ch.frequency / s.maxFreq)
        * 80 + 1.5));
446     const ampHeight = (H / 2 - 14) * (ch.amplitude / s.maxAmp);
447
448     ctx.beginPath();
449     for (let x = 0; x <= W; x++) {
450         const t = (x / W) * cycles * Math.PI * 2 + phase;
451         const py = H / 2 - computeY(ch.waveType, t) * ampHeight;
452         x === 0 ? ctx.moveTo(x, py) : ctx.lineTo(x, py);
453     }
454     ctx.strokeStyle = ch.color + '55'; ctx.lineWidth = 6; ctx.
        shadowColor = ch.color; ctx.shadowBlur = 20; ctx.stroke();
455
456     ctx.shadowBlur = 0; ctx.beginPath();
457     for (let x = 0; x <= W; x++) {
458         const t = (x / W) * cycles * Math.PI * 2 + phase;
459         const py = H / 2 - computeY(ch.waveType, t) * ampHeight;
460         x === 0 ? ctx.moveTo(x, py) : ctx.lineTo(x, py);
461     }

```

```

462     ctx.strokeStyle = ch.color; ctx.lineWidth = 2; ctx.shadowColor = ch
        .color; ctx.shadowBlur = 8; ctx.stroke(); ctx.shadowBlur = 0;
463 }
464
465 function draw() {
466     const W = canvas.offsetWidth, H = canvas.offsetHeight;
467     if (!W || !H) { animId = requestAnimationFrame(draw); return; }
468     ctx.fillStyle = '#06090e'; ctx.fillRect(0, 0, W, H);
469     drawGrid(W, H); drawWaveform(W, H, State.getActive());
470     phase += 0.045; animId = requestAnimationFrame(draw);
471 }
472
473     return { start: draw };
474 })();
475
476 /*
477     MODULE: SliderUtil
478
479     */
479 const SliderUtil = {
480     updateFill(el) {
481         const pct = ((el.value - el.min) / (el.max - el.min)) * 100;
482         el.style.setProperty('--pct', `${Math.max(0, pct)}%`);
483     }
484 };
485
486 /*
487     MODULE: UI
488
489     */
489 const UI = (() => {
490
491     function setFreqDisplay(hz, color) {
492         const { num, unit } = Format.freqLabel(hz);
493         const fi = document.getElementById('freqInput');
494         const fu = document.getElementById('freqUnit');
495         if (fi && document.activeElement !== fi) fi.value = num;
496         if (fu && document.activeElement !== fu) fu.value = unit;
497         document.getElementById('freqDisplayWrap').style.color = color;
498         document.getElementById('freqSliderVal').textContent = Format.freq(
            hz);
499     }
500

```

```

501 function setAmpDisplay(v, color) {
502     const { num, unit } = Format.ampLabel(v);
503     const ai = document.getElementById('ampInput');
504     const au = document.getElementById('ampUnit');
505     if (ai && document.activeElement !== ai) ai.value = num;
506     if (au) au.textContent = unit;
507     document.getElementById('ampDisplayWrap').style.color = color;
508     document.getElementById('ampSliderVal').textContent = Format.amp(v)
509     ;
510 }
511
512 function updateSummary() {
513     const s = State.get();
514     const c1 = s.channels[1], c2 = s.channels[2];
515     document.getElementById('summaryCh1').textContent = `${Format.freq(
516         c1.frequency)}    ${Format.amp(c1.amplitude)}';
517     document.getElementById('summaryCh2').textContent = `${Format.freq(
518         c2.frequency)}    ${Format.amp(c2.amplitude)}';
519 }
520
521 function setWaveBtns(waveType, chId) {
522     const cls = 'active-ch${chId}';
523     ['sine', 'square', 'triangular'].forEach(t => {
524         document.getElementById('btn-${t}').className = 'wave-btn' + (t
525             === waveType ? ' ${cls}' : '');
526     });
527 }
528
529 function updateCSSActiveVars(color, dimColor, glowColor) {
530     const root = document.documentElement.style;
531     root.setProperty('--active', color);
532     root.setProperty('--active-dim', dimColor);
533     root.setProperty('--active-glow', glowColor);
534 }
535
536 /*          Public API          */
537
538 function switchChannel(ch) {
539     if (State.getActiveId() !== ch) {
540         Network.send('setActiveChannel', { ch: ch });
541     }
542
543     State.setActiveId(ch);
544     const chData = State.getChannel(ch);
545     const s = State.get();
546     const color = chData.color, isDimColor = color + '22', isGlowColor
547         = color + '55';

```

```

543
544     updateCSSActiveVars(color, isDimColor, isGlowColor);
545
546     document.getElementById('tab1').className = 'tab-btn' + (ch === 1 ?
547         ' active-ch1' : '');
548     document.getElementById('tab2').className = 'tab-btn' + (ch === 2 ?
549         ' active-ch2' : '');
550
551     document.getElementById('chDot').style.cssText = 'background:${
552         color};box-shadow:0 0 8px ${color}';
553     document.getElementById('chName').textContent = 'Channel ${ch}';
554
555     const waveNames = { sine: 'Sine Wave', square: 'Square Wave',
556         triangular: 'Triangle Wave' };
557     document.getElementById('chWaveBadge').textContent = waveNames[
558         chData.waveType];
559     document.getElementById('cornerTL').textContent = 'CH${ch}';
560     document.getElementById('cornerTR').textContent = chData.waveType.
561         toUpperCase();
562
563     const sliderClass = ch === 1 ? 'slider-green' : 'slider-orange';
564     const fs = document.getElementById('freqSlider'), as = document.
565         getElementById('ampSlider');
566     fs.className = sliderClass; as.className = sliderClass;
567     fs.value = chData.frequency; as.value = Math.round((chData.
568         amplitude / s.maxAmp) * State.AMP_STEPS);
569     SliderUtil.updateFill(fs); SliderUtil.updateFill(as);
570
571     setWaveBtns(chData.waveType, ch);
572     setFreqDisplay(chData.frequency, color);
573     setAmpDisplay(chData.amplitude, color);
574     updateSummary();
575 }
576
577 function onFreqChange(val) {
578     const hz = Number(val);
579     const ch = State.getActiveId();
580     State.setFreq(ch, hz);
581     setFreqDisplay(hz, State.getActive().color);
582     SliderUtil.updateFill(document.getElementById('freqSlider'));
583     updateSummary();
584
585     // Notify backend
586     Network.sendDebounce('setFreq', { ch: ch, val: hz });
587 }
588
589 function onAmpChange(val) {
590     const s = State.get();

```

```

582     const amp = parseFloat(((Number(val) / State.AMP_STEPS) * s.maxAmp)
583         .toFixed(1));
584     const ch = State.getActiveId();
585     State.setAmp(ch, amp);
586     setAmpDisplay(amp, State.getActive().color);
587     SliderUtil.updateFill(document.getElementById('ampSlider'));
588     updateSummary();
589     // Notify backend
590     Network.sendDebounce('setAmp', { ch: ch, val: amp });
591 }
592
593 function setWave(type) {
594     const ch = State.getActiveId();
595     if (State.getChannel(ch).waveType !== type) {
596         Network.send('setWave', { ch: ch, type: type });
597     }
598
599     State.setWave(ch, type);
600     setWaveBtns(type, ch);
601     const waveNames = { sine: 'Sine Wave', square: 'Square Wave',
602         triangular: 'Triangle Wave' };
603     document.getElementById('chWaveBadge').textContent = waveNames[type];
604     document.getElementById('cornerTR').textContent = type.toUpperCase();
605 }
606
607 function onFreqInput(valStr) {
608     let unit = document.getElementById('freqUnit').value;
609     let num = parseFloat(valStr);
610     if (isNaN(num)) return;
611     let hz = num;
612     if (unit === 'kHz') hz *= 1e3;
613     else if (unit === 'MHz') hz *= 1e6;
614
615     const maxFreq = State.get().maxFreq;
616     hz = Math.max(10, Math.min(hz, maxFreq));
617     const ch = State.getActiveId();
618     State.setFreq(ch, hz);
619
620     setFreqDisplay(hz, State.getActive().color);
621     const fs = document.getElementById('freqSlider');
622     fs.value = hz;
623     SliderUtil.updateFill(fs);
624     updateSummary();

```

```

625     // Notify backend
626     Network.send('setFreq', { ch: ch, val: hz });
627 }
628
629 function onFreqUnitChange(unitValue) {
630     onFreqInput(document.getElementById('freqInput').value);
631 }
632
633 function onAmpInput(valStr) {
634     let num = parseFloat(valStr);
635     if (isNaN(num)) return;
636     const maxAmp = State.get().maxAmp;
637     let amp = Math.max(0, Math.min(num, maxAmp));
638     const ch = State.getActiveId();
639     State.setAmp(ch, amp);
640
641     setAmpDisplay(amp, State.getActive().color);
642     const as = document.getElementById('ampSlider');
643     as.value = Math.round((amp / maxAmp) * State.AMP_STEPS);
644     SliderUtil.updateFill(as);
645     updateSummary();
646
647     // Notify backend
648     Network.send('setAmp', { ch: ch, val: amp });
649 }
650
651 async function syncWithHardware() {
652     const data = await Network.getStatus();
653     if (!data) return;
654
655     // 1. Update internal state
656     const s = State.get();
657     s.activeChannel = data.activeChannel;
658
659     for (let id in data.channels) {
660         State.setFreq(id, data.channels[id].frequency);
661         State.setAmp(id, data.channels[id].amplitude);
662         State.setWave(id, data.channels[id].waveType);
663     }
664
665     // 2. Refresh the UI elements
666     // We only refresh if the user isn't currently typing/dragging
667     if (document.activeElement.tagName !== 'INPUT') {
668         UI.switchChannel(s.activeChannel);
669     }
670 }
671

```

```

672     return { switchChannel, onFreqChange, onAmpChange, setWave,
673             onFreqInput, onFreqUnitChange, onAmpInput, syncWithHardware };
674 })();
675 /*
676     INIT
677     */
678 (function init() {
679     SliderUtil.updateFill(document.getElementById('freqSlider'));
680     SliderUtil.updateFill(document.getElementById('ampSlider'));
681     Oscilloscope.start();
682     setInterval(() => UI.syncWithHardware(), 1000);
683 })();
684 </script>
685 </body>
686 </html>

```

### 4.9.3 Web Server Backend

Listing 10: ESP32 backend implementation (sketch.ino)

```

1
2 #include <WiFi.h>
3 #include <WebServer.h>
4
5 /*
6     APP LOGIC & STATE
7     */
8 enum WaveformType { WAVE_SINE, WAVE_SQUARE, WAVE_TRIANGLE };
9
10 struct ChannelState {
11     float frequency;
12     float amplitude;
13     WaveformType waveform;
14     bool enabled;
15 };
16
17 struct AppState {
18     int active_channel;
19     ChannelState ch[2];
20 };

```

```

21
22 AppState globalState = {
23     0,
24     {
25         {10000.00, 5.00, WAVE_SINE, true},
26         {1000.00, 3.50, WAVE_SINE, true}
27     }
28 };
29
30 // Hardware control functions (Updates serial monitor)
31 void app_set_frequency(int ch, float freq) {
32     globalState.ch[ch].frequency = freq;
33     Serial.printf("[HARDWARE]_CH%d_Frequency_set_to: %.2f_Hz\n", ch+1,
34         freq);
35 }
36 void app_set_amplitude(int ch, float amp) {
37     globalState.ch[ch].amplitude = amp;
38     Serial.printf("[HARDWARE]_CH%d_Amplitude_set_to: %.2f_V\n", ch+1,
39         amp);
40 }
41 void app_set_waveform(int ch, WaveformType w) {
42     globalState.ch[ch].waveform = w;
43     const char* names[] = {"SINE", "SQUARE", "TRIANGLE"};
44     Serial.printf("[HARDWARE]_CH%d_Waveform_set_to: %s\n", ch+1, names[
45         w]);
46 }
47 void app_set_enable(int ch, bool en) {
48     globalState.ch[ch].enabled = en;
49     Serial.printf("[HARDWARE]_CH%d_State: %s\n", ch+1, en ? "ENABLED" :
50         "DISABLED");
51 }
52 void app_set_active_channel(int ch) {
53     globalState.active_channel = ch;
54     Serial.printf("[UI]_Switched_to_Channel_%d\n", ch+1);
55 }
56 const AppState* app_get_state() { return &globalState; }
57
58 /*
59
60     WEB SERVER & UI CONTENT
61
62     */
63 const char* ssid = "ESP32-Generator";
64 const char* password = "password123";
65
66 WebServer server(80);

```

```

61
62 const char INDEX_HTML[] PROGMEM = R"rawliteral(
63 <!DOCTYPE_<html>
64 <html_<head>
65 <meta_charset="UTF-8">
66 <meta_name="viewport">content="width=device-width, initial-scale=1.0"
67 />
68 <title>Dual_Channel_Function_Generator</title>
69 <link_href="https://fonts.googleapis.com/css2?family=Space+Mono:
    wght@400;700&family=Syne:wght@400;600;700;800&display=swap" rel="
    stylesheet"/>
70 <style>
71 *, *::before, *::after { box-sizing: border-box; margin: 0; padding
    : 0; }
72 :root {
73     --radius-md: 10px; --radius-xl: 22px;
74     --bg: #080b10; --surface: #0e1319; --surface-2: #131920; --border
        : #1c2330; --border-light: #2a3545;
75     --ch1: #00ffaa; --ch1-dim: #00ffaa22; --ch1-glow: #00ffaa55;
76     --ch2: #ff6b35; --ch2-dim: #ff6b3522; --ch2-glow: #ff6b3555;
77     --text-primary: #dde8f5; --text-sec: #6b7f99; --text-muted:
        #384556;
78     --font-ui: 'Syne', sans-serif; --font-mono: 'Space_Mono',
        monospace;
79     --active: var(--ch1); --active-dim: var(--ch1-dim); --active-glow
        : var(--ch1-glow);
80 }
81 body { background: var(--bg); font-family: var(--font-ui); color:
    var(--text-primary); padding: 20px 16px; background-image:
    linear-gradient(rgba(0,255,170,.01) 1px, transparent 1px),
    linear-gradient(90deg, rgba(0,255,170,.01) 1px, transparent 1px)
    ; background-size: 40px 40px; }
82 .app { width: 100%; max-width: 500px; margin: 0 auto; }
83 .header { display: flex; align-items: center; gap: 16px; padding:
    20px; background: var(--surface); border: 1px solid var(--border
    ); border-radius: var(--radius-xl); margin-bottom: 14px; }
84 .header-icon { width: 40px; height: 40px; border-radius: 8px;
    background: var(--ch1); display: flex; align-items: center;
    justify-content: center; font-size: 20px; box-shadow: 0 0 20px
    var(--ch1-glow); }
85 .tab-bar { display: grid; grid-template-columns: 1fr 1fr; gap: 8px;
    margin-bottom: 14px; }
86 .tab-btn { padding: 14px; border-radius: var(--radius-md); border:
    1px solid var(--border-light); background: var(--surface); color
    : var(--text-sec); font-weight: 800; cursor: pointer; display:
    flex; align-items: center; justify-content: center; gap: 8px;

```

```

transition: 0.2s; }
87 .tab-btn.active-ch1 { background: var(--ch1-dim); color: var(--ch1)
    ; border-color: var(--ch1); }
88 .tab-btn.active-ch2 { background: var(--ch2-dim); color: var(--ch2)
    ; border-color: var(--ch2); }
89 .tab-dot { width: 8px; height: 8px; border-radius: 50%; background:
    currentColor; }
90 .ch-panel { background: var(--surface); border: 1px solid var(--
    border); border-radius: var(--radius-xl); overflow: hidden; }
91 .ch-panel-header { display: flex; align-items: center; justify-
    content: space-between; padding: 14px 20px; background: var(--
    surface-2); border-bottom: 1px solid var(--border); }
92 .pwr-btn { padding: 6px 12px; border-radius: 99px; border: 1px
    solid var(--border-light); background: var(--bg); color: var(--
    text-sec); font-family: var(--font-mono); font-size: 10px;
    cursor: pointer; }
93 .pwr-btn.on { background: var(--active-dim); color: var(--active);
    border-color: var(--active-glow); }
94 .canvas-wrap { padding: 10px; background: #05070a; position:
    relative; }
95 .canvas-corner { position: absolute; font-family: var(--font-mono);
    font-size: 9px; color: var(--text-muted); padding: 10px; z-
    index: 2; }
96 .tr { right: 0; top: 0; } .tl { left: 0; top: 0; }
97 canvas { width: 100%; height: 160px; display: block; }
98 .stats-grid { display: grid; grid-template-columns: 1fr 1fr; border
    -bottom: 1px solid var(--border); }
99 .stat-cell { padding: 16px 20px; border-right: 1px solid var(--
    border); }
100 .stat-label { font-family: var(--font-mono); font-size: 10px; color
    : var(--text-sec); text-transform: uppercase; margin-bottom: 4px
    ; }
101 .stat-value { display: flex; align-items: baseline; font-family:
    var(--font-mono); font-size: 22px; font-weight: 700; color: var
    (--active); }
102 .stat-input { background: transparent; border: none; color: inherit
    ; font: inherit; width: 100%; outline: none; }
103 .stat-select { background: transparent; border: none; color:
    inherit; font: inherit; font-size: 12px; cursor: pointer;
    outline: none; padding-left: 4px; }
104 .controls-section { padding: 20px; display: flex; flex-direction:
    column; gap: 20px; }
105 .wave-btns { display: grid; grid-template-columns: repeat(3, 1fr);
    gap: 8px; }
106 .wave-btn { padding: 12px; border-radius: var(--radius-md); border:
    1px solid var(--border-light); background: var(--bg); color:
    var(--text-sec); cursor: pointer; display: flex; flex-direction:

```

```

        column; align-items: center; gap: 8px; font-size: 11px; font-
weight: 700; }
107 .wave-btn svg { width: 30px; height: 18px; stroke: currentColor;
    fill: none; stroke-width: 2; }
108 .wave-btn.active-ch1 { background: var(--ch1-dim); color: var(--ch1
    ); border-color: var(--ch1); }
109 .wave-btn.active-ch2 { background: var(--ch2-dim); color: var(--ch2
    ); border-color: var(--ch2); }
110 input[type=range] { -webkit-appearance: none; width: 100%; height:
    4px; border-radius: 2px; outline: none; margin: 15px 0;
    background: var(--border-light); }
111 input[type=range]::-webkit-slider-thumb { -webkit-appearance: none;
    width: 18px; height: 18px; border-radius: 50%; background: var
    (--bg); border: 2px solid var(--active); box-shadow: 0 0 10px
    var(--active-glow); cursor: pointer; }
112 .slider-current { float: right; font-family: var(--font-mono); font
    -size: 11px; color: var(--active); }
113 .summary-bar { display: grid; grid-template-columns: 1fr 1fr; gap:
    10px; margin-top: 14px; }
114 .summary-chip { background: var(--surface); border: 1px solid var
    (--border); border-radius: var(--radius-md); padding: 12px;
    display: flex; align-items: center; gap: 8px; font-size: 10px; }
115 .disabled { opacity: 0.25; pointer-events: none; }
116 </style>
117 </head>
118 <body>
119 <div class="app">
120   <div class="header">
121     <div class="header-icon">   </div>
122     <div><h1 style="font-size:16px">FUNCTION GENERATOR</h1><p style="
        font-size:10px; color:var(--text-sec)">10Hz - 1MHz DUAL CHANNEL
        </p></div>
123   </div>
124   <div class="tab-bar">
125     <button id="tab1" class="tab-btn" onclick="UI.switchChannel(1)"><
        span class="tab-dot"></span>CH 1</button>
126     <button id="tab2" class="tab-btn" onclick="UI.switchChannel(2)"><
        span class="tab-dot"></span>CH 2</button>
127   </div>
128   <div class="ch-panel">
129     <div class="ch-panel-header">
130       <div style="font-weight:800; font-size:13px" id="chTitle">CHANNEL
        1</div>
131       <button class="pwr-btn" id="pwrBtn" onclick="UI.togglePower()">
        POWER: ON</button>
132     </div>
133   <div class="canvas-wrap">

```

```

134     <div class="canvas-corner_t1" id="cornerTL">CH1</div><div class="
        canvas-corner_tr" id="cornerTR">SINE</div>
135     <canvas id="waveCanvas"></canvas>
136 </div>
137 <div id="mainControls">
138     <div class="stats-grid">
139         <div class="stat-cell">
140             <div class="stat-label">Frequency</div>
141             <div class="stat-value"><input type="number" id="freqInput"
                step="0.01" class="stat-input" onchange="UI.onFreqInput(
                this.value)" /><select id="freqUnit" class="stat-select"
                onchange="UI.onFreqUnitChange()"><option>Hz</option><
                option>kHz</option><option>MHz</option></select></div>
142         </div>
143         <div class="stat-cell">
144             <div class="stat-label">Amplitude</div>
145             <div class="stat-value"><input type="number" id="ampInput"
                step="0.01" class="stat-input" onchange="UI.onAmpInput(
                this.value)" /><select id="ampUnit" class="stat-select"
                onchange="UI.onAmpUnitChange()"><option>V</option><option>
                mV</option></select></div>
146         </div>
147     </div>
148     <div class="controls-section">
149         <div class="wave-btns">
150             <button class="wave-btn" id="btn-sine" onclick="UI.setWave('
                sine')"><svg viewBox="0 0 36 20"><path d="M2 10 C6 2, 10
                2, 13 10 C16 18, 20 18, 23 10 C26 2, 30 2, 34 10"/></svg>
                SINE</button>
151             <button class="wave-btn" id="btn-square" onclick="UI.setWave
                ('square')"><svg viewBox="0 0 36 20"><path d="M2 15 L2 5
                L12 5 L12 15 L22 15 L22 5 L32 5 L32 15 L34 15"/></svg>
                SQUARE</button>
152             <button class="wave-btn" id="btn-triangular" onclick="UI.
                setWave('triangular')"><svg viewBox="0 0 36 20"><path d="
                M2 15 L9 5 L18 15 L27 5 L34 15"/></svg>TRI</button>
153         </div>
154         <div>
155             <div class="stat-label">Frequency <span class="slider-current
                " id="curFreq"></span></div>
156             <input type="range" id="freqSlider" min="10" max="1000000"
                oninput="UI.onFreqChange(this.value)" />
157         </div>
158         <div>
159             <div class="stat-label">Amplitude <span class="slider-current
                " id="curAmp"></span></div>

```

```

160         <input type="range" id="ampSlider" min="0" max="1000" oninput
           ="UI.onAmpChange(this.value)" />
161     </div>
162 </div>
163 </div>
164 </div>
165 <div class="summary-bar">
166     <div class="summary-chip" id="sum1">CH1: --</div>
167     <div class="summary-chip" id="sum2">CH2: --</div>
168 </div>
169 </div>
170 <script>
171 const Network = {
172     send(ep, p={}) { fetch('/${ep}?${new URLSearchParams(p)}').catch(e=>
        console.error(e)); },
173     async getStatus() { try { const r=await fetch('/getStatus'); return
        await r.json(); } catch(e) { return null; } }
174 };
175 const State = {
176     active: 1, maxF: 1000000, minF: 10, maxA: 10,
177     ch: { 1: { f: 10000.00, a: 5.00, w: 'sine', c: '#00ffaa', en: true },
        2: { f: 1000.00, a: 3.50, w: 'sine', c: '#ff6b35', en: true } }
178 };
179 const UI = {
180     update() {
181         const c = State.ch[State.active];
182         const root = document.documentElement;
183         root.style.setProperty('--active', State.active === 1 ? 'var(--ch1)
        ' : 'var(--ch2)');
184         root.style.setProperty('--active-dim', State.active === 1 ? 'var(--
        ch1-dim)' : 'var(--ch2-dim)');
185         root.style.setProperty('--active-glow', State.active === 1 ? 'var
        (--ch1-glow)' : 'var(--ch2-glow)');
186         document.getElementById('tab1').className = 'tab-btn' + (State.
        active === 1 ? '▢active-ch1' : '');
187         document.getElementById('tab2').className = 'tab-btn' + (State.
        active === 2 ? '▢active-ch2' : '');
188         document.getElementById('chTitle').textContent = "CHANNEL▢" + State
        .active;
189         document.getElementById('cornerTL').textContent = "CH" + State.
        active;
190         document.getElementById('cornerTR').textContent = c.w.toUpperCase()
        ;
191         document.getElementById('pwrBtn').textContent = "POWER:▢" + (c.en ?
        "ON" : "OFF");
192         document.getElementById('pwrBtn').className = "pwr-btn" + (c.en ? "
        ▢on" : "");

```

```

193     document.getElementById('mainControls').className = c.en ? "" : "
        disabled";
194
195     // Freq Format
196     let fv = c.f; let fu = "Hz";
197     if(fv >= 1e6) { fv /= 1e6; fu = "MHz"; } else if(fv >= 1e3) { fv /=
        1e3; fu = "kHz"; }
198     document.getElementById('freqInput').value = fv.toFixed(2);
199     document.getElementById('freqUnit').value = fu;
200     document.getElementById('curFreq').textContent = c.f.toFixed(2) + "
        Hz";
201
202     // Amp Format
203     let av = c.a; let au = document.getElementById('ampUnit').value;
204     if(au === 'mV') av *= 1000;
205     document.getElementById('ampInput').value = av.toFixed(2);
206     document.getElementById('curAmp').textContent = c.a.toFixed(2) + "
        V";
207
208     document.getElementById('freqSlider').value = c.f;
209     document.getElementById('ampSlider').value = (c.a / State.maxA) *
        1000;
210     ['sine', 'square', 'triangular'].forEach(w => {
211         document.getElementById('btn-' + w).className = 'wave-btn' + (c.w
            === w ? ' active-ch${State.active}' : '');
212     });
213     for(let i=1; i<=2; i++){
214         const s = State.ch[i];
215         document.getElementById('sum'+i).textContent = 'CH${i}: ${s.en ?
            s.f.toFixed(2)+'Hz' : 'OFF'}';
216         document.getElementById('sum'+i).style.color = s.en ? State.ch[i
            ].c : 'var(--text-muted)';
217     }
218 },
219 switchChannel(ch) { State.active = ch; Network.send('setActiveChannel
    ', {ch}); this.update(); },
220 togglePower() { const c = State.ch[State.active]; c.en = !c.en;
    Network.send('setEnabled', {ch: State.active, val: c.en?1:0}); this
        .update(); },
221 onFreqChange(v) { let hz = Math.max(State.minF, Math.min(State.maxF,
    Number(v))); State.ch[State.active].f = hz; Network.send('setFreq'
    , {ch: State.active, val: hz.toFixed(2)}); this.update(); },
222 onFreqInput(v) { let hz = parseFloat(v); let u = document.
    getElementById('freqUnit').value; if(u === 'kHz') hz *= 1000; else
        if(u === 'MHz') hz *= 1000000; this.onFreqChange(hz); },
223 onFreqUnitChange() { this.onFreqInput(document.getElementById('
    freqInput').value); },

```

```

224   onAmpChange(v) { let amp = (v/1000) * State.maxA; State.ch[State.
      active].a = amp; Network.send('setAmp', {ch: State.active, val:
      amp.toFixed(2)}); this.update(); },
225   onAmpInput(v) { let val = parseFloat(v); if(document.getElementById('
      ampUnit').value === 'mV') val /= 1000; this.onAmpChange((val/State
      .maxA)*1000); },
226   onAmpUnitChange() { this.onAmpInput(document.getElementById('ampInput
      ').value); },
227   setWave(type) { State.ch[State.active].w = type; Network.send('
      setWave', {ch: State.active, type}); this.update(); }
228 };
229 const Oscilloscope = {
230   canvas: document.getElementById('waveCanvas'), phase: 0,
231   start() { this.ctx = this.canvas.getContext('2d'); window.onresize =
      ()=>this.resize(); this.resize(); this.draw(); },
232   resize() { this.canvas.width = this.canvas.offsetWidth; this.canvas.
      height = this.canvas.offsetHeight; },
233   draw() {
234     const {ctx, canvas:v} = this; const {width:w, height:h} = v; const
      c = State.ch[State.active];
235     ctx.clearRect(0,0,w,h);
236     ctx.strokeStyle = '#1c2330'; ctx.lineWidth = 1;
237     for(let i=0; i<=8; i++) { ctx.beginPath(); ctx.moveTo(i*w/8, 0);
      ctx.lineTo(i*w/8, h); ctx.stroke(); }
238     for(let i=0; i<=4; i++) { ctx.beginPath(); ctx.moveTo(0, i*h/4);
      ctx.lineTo(w, i*h/4); ctx.stroke(); }
239     if(c.en) {
240       ctx.beginPath(); ctx.strokeStyle = c.c; ctx.lineWidth = 3; ctx.
      shadowBlur = 10; ctx.shadowColor = c.c;
241       // Cycles based on frequency (wavelength changes)
242       let visibleCycles = 1 + (Math.log10(c.f) - 1) * 2;
243       for(let x=0; x<w; x++) {
244         let t = (x/w) * Math.PI * 2 * visibleCycles + this.phase;
245         let y=0;
246         if(c.w==='sine') y = Math.sin(t);
247         else if(c.w==='square') y = Math.sin(t) >= 0 ? 0.7 : -0.7;
248         else y = (Math.abs((t % (Math.PI*2)) - Math.PI) / Math.PI) * 2
          - 1;
249         const py = h/2 - y*(h/3)*(c.a/State.maxA);
250         x===0 ? ctx.moveTo(x,py) : ctx.lineTo(x,py);
251       }
252       ctx.stroke(); ctx.shadowBlur = 0;
253       this.phase += 0.05; // Fixed animation speed
254     }
255     requestAnimationFrame(()=>this.draw());
256   }
257 };

```

```

258 Oscilloscope.start();
259 setInterval(async () => {
260     const s = await Network.getStatus();
261     if(s && document.activeElement.tagName !== 'INPUT') {
262         State.active = s.activeChannel;
263         for(let i=1; i<=2; i++) {
264             State.ch[i].f = s.channels[i].frequency;
265             State.ch[i].a = s.channels[i].amplitude;
266             State.ch[i].w = s.channels[i].waveType;
267             State.ch[i].en = s.channels[i].enabled;
268         }
269         UI.update();
270     }
271 }, 2000);
272 UI.update();
273 </script>
274 </body>
275 </html>
276 )rawliteral";
277
278 /*
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300

```

```

301  server.send(200, "text/plain", "OK");
302  }
303
304  void handleSetWave() {
305      int ch = server.arg("ch").toInt() - 1;
306      String t = server.arg("type");
307      WaveformType w = WAVE_SINE;
308      if (t == "square") w = WAVE_SQUARE; else if (t == "triangular") w =
          WAVE_TRIANGLE;
309      if (ch >= 0 && ch < 2) app_set_waveform(ch, w);
310      server.send(200, "text/plain", "OK");
311  }
312
313  void handleSetActiveChannel() {
314      int ch = server.arg("ch").toInt() - 1;
315      if (ch >= 0 && ch < 2) app_set_active_channel(ch);
316      server.send(200, "text/plain", "OK");
317  }
318
319  void handleGetStatus() {
320      const AppState* s = app_get_state();
321      String j = "{" + String(s->active_channel + 1) + "
          , \"channels\": {";
322      for (int i = 0; i < 2; i++) {
323          j += "\"" + String(i + 1) + "\": {" + String(s->ch[i].
              frequency, 2) + ", \"amplitude\": " + String(s->ch[i].amplitude,
              2) + ", \"enabled\": " + String(s->ch[i].enabled ? "true" : "false") +
              ", \"waveType\": \"";
324          if (s->ch[i].waveform == WAVE_SQUARE) j += "square"; else if (s->ch[i].
              waveform == WAVE_TRIANGLE) j += "triangular"; else j += "sine";
325          j += "\"}"; if (i == 0) j += ", ";
326      }
327      j += "}}";
328      server.send(200, "application/json", j);
329  }
330
331  void setup() {
332      Serial.begin(115200);
333      WiFi.softAP(ssid, password);
334      Serial.print("AP Started. IP: "); Serial.println(WiFi.softAPIP());
335
336      server.on("/", handleRoot);
337      server.on("/setFreq", handleSetFreq);
338      server.on("/setAmp", handleSetAmp);
339      server.on("/setWave", handleSetWave);
340      server.on("/setEnabled", handleSetEnable);
341      server.on("/setActiveChannel", handleSetActiveChannel);

```

```

342     server.on("/getStatus", handleGetStatus);
343     server.begin();
344 }
345
346 void loop() {
347     server.handleClient();
348 }

```

## 4.10 Techniques and Implementation

The techniques used during the design and development of the prototype are summarized in Table 3. This table outlines the major fabrication and assembly methods used in building the prototype.

Table 3: Techniques used during prototype development

| Technique        | Description                                                                | Implementation Details                                                                                                  |
|------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Laser Cutting    | Used to precisely cut the acrylic sheets according to the required design. | The design net was created in RDWorks software and then laser cut on acrylic sheets.                                    |
| Heat Bending     | Used to bend acrylic sheets to achieve the required structural shape.      | A heat gun was applied along the bending lines to soften the acrylic, allowing it to be folded into the desired angles. |
| Chemical Sealing | Used to permanently join acrylic pieces together.                          | Chloroform was applied along the joints to chemically bond the acrylic surfaces and seal the structure.                 |

# Minutes of Meeting

**Date:** 13/01/2026

**Topic:** Design Direction Clarification & Task Alignment

## Attendees

- Teaching Assistant
- Project Team Members

## Agenda

1. Clarify design direction and align tasks
2. Decide design changes for current phase
3. Finalize workflow and tools for CAD and PCB design

## Discussion

**Design Change:** It was decided to replace the power supply module with a function generator for the current design phase.

### **Function Generator Study:**

- The team will analyze a working function generator to identify key functional and design features.
- Based on this study, the team will attempt to replicate the essential characteristics in the proposed design.

### **Design Tools & Workflow:**

#### **CAD and Laser Cutting:**

- RDWorks will be used for preparing 2D vector files for laser cutting.
- AutoCAD will be used for detailed mechanical design.

#### **PCB Design:**

- iCAD or EasyEDA will be used for schematic and PCB layout.
- User-contributed components will be avoided.
- SnapEDA will be used to import verified component footprints and schematics.

**Component Planning:** A component requirement list (BoM) will be prepared in the following format:

| Sl. No. | Component Name | Quantity |
|---------|----------------|----------|
|---------|----------------|----------|

The quantity planning will:

- cover all essential components,
- account for components susceptible to damage during assembly/testing,
- include an additional 20% buffer inventory to ensure availability.

## Action Items

- Finalize requirements for Friday submission.
- Ensure consistency between mechanical design and PCB layout.

# Minutes of Meeting

**Date:** 13/01/2026

**Topic:** Function Generator: Requirements, Reference Designs & Improvements

## Attendees

- Teaching Assistant
- Hardware Team

## Agenda

1. Define requirements for the function generator
2. Review reference designs (commercial and previous-year student builds)
3. Identify possible improvements and documentation requirements

## Discussion

### Requirements:

- The function generator should support operation up to 2.5 MHz.
- Final specifications (waveforms, resolution, accuracy, interface, etc.) are to be finalized based on feasibility.
- The design must be usable in the ELP101 lab environment.

### Reference Designs Shown:

- One commercially available working function generator
- Two function generators built by last year's student teams

### Commercial Working Function Generator (Observations):

#### Mechanical / Design Features:

- Openable design with a rear section that pops out for access and maintenance
- Metallic enclosure with ventilation holes
- Stands to allow the device to be tilted for better readability

Suggested improvement: add an additional inclined surface and determine the optimal viewing angle for clearer observation.

**Power & Internal Architecture:**

- Input: AC single-phase
- Internal supply: 3.3 V DC using an AC–DC converter, along with a step-down stage
- Control structure:
  - Microchip-manufactured microcontroller for handling buttons/user input
  - Communication to an FPGA (waveform generator) via ribbon cable

**Output & Protection:**

- Output through BNC connectors
- Protection/isolation approaches discussed:
  - Physical isolation
  - Magnetic isolation
  - Relay-based switching
  - Opto-isolators (LED triggers photodiode; 4-pin and 6-pin variants)

**Previous Year Student Designs (Observations):**

**Power Input Approach:**

- No transformer used at the input.
- Since each ELP101 table provides a power supply, designs directly used:
  - +15 V, −15 V, and GND (clearly labeled)

**Core Components & Wave Generation:**

- ESP32 used as the main controller in both designs
- One design used USB Micro-B input
- Wave generation methods:
  - One team used AD9833
  - Another used the ESP32 internal 8-bit DAC
- Resolution note: higher resolution ( $\sim 12$ -bit) provides improved waveform quality

### **Interface & Display:**

- Knobs used for amplitude control
- Control methods:
  - Wi-Fi based control (website + source code provided)
  - Joystick-based control
- Display types used:
  - OLED
  - Alphanumeric display

### **Analog Stage Notes:**

- Op-amps used for amplification
- $-15\text{ V}$  used for op-amp supply requirements (VDD/Vsat considerations)
- Op-amp bandwidth was highlighted as important:
  - 741 may not be suitable
  - Higher bandwidth op-amps (example discussed:  $\sim 15\text{ MHz}$ ) were used in successful designs
- Construction approach: modular design, with clear labeling of pins and polarity

### **Improvements Suggested:**

- Add an LCD display to show the generated waveform
- Explore making the design handheld

**End Goal:** Build a function generator that is reliable and practically usable in ELP101.

**Documentation Requirement:** A user manual must be prepared clearly stating:

- Supported waveforms
- Maximum achievable frequency
- Operating limitations and performance claims

### **Cost Consideration:**

- ESP32 development board cost:  $\sim 800$
- ESP32 chip cost:  $\sim 200$
- Cost optimization not to be considered in final design decisions

## Action Items

- Finalize feasibility-based specifications.
- Document waveforms, max frequency, and operating limitations in the user manual.

# Minutes of Meeting

**Date:** 13/01/2026

**Topic:** Software Team Formation, Module Planning & System Architecture

## Attendees

- Software Team Members

## Agenda

1. Finalize team structure for parallel development
2. Discuss microcontroller and display options
3. Plan modular architecture and integration strategy

## Discussion

### Team Structure Finalized:

- Display Team – screen selection and UI/UX planning
- Signal Processing Team – waveform handling, filtering, and output quality improvements
- Wi-Fi Control Team – wireless control interface and connectivity features
- Software Research Team – exploring new ideas/tools and coordinating with the main development team

### Key Discussion Points:

- Discussion on microcontroller type and functionality based on requirements and feasibility
- Options for screen/display were discussed considering readability and integration
- Emphasis on modular system development for efficient testing and integration

## Action Items

- Teams will work on their respective modules and share progress regularly.
- Final decisions on microcontroller and display will be taken after feasibility comparison.

# Minutes of Meeting

**Date:** 13/01/2026

**Topic:** CAD Team Formation, Laser Cutting Basics & Enclosure Planning

## Attendees

- Teaching Assistant
- CAD Team Members

## Agenda

1. Introduce CAD workflow and tools
2. Demonstrate laser cutting basics
3. Plan enclosure design approach

## Discussion

### CAD Basics Demonstration:

- The TA demonstrated the basic CAD workflow required for the project.
- A sample engraving (“TUESDAY”) was performed using the laser machine.

### Laser Machine Demonstration:

- Power settings depend on the design/material being used.
- Operating procedure:
  - Open files
  - Verify the design layout and alignment
  - Set the origin before starting the operation

## Action Items

- Design and finalize the enclosure keeping usability, modularity, and manufacturability in mind.

# Minutes of Meeting

**Date:** 20/01/2026

**Topic:** Hardware Architecture & Roadmap (Platform Evaluation)

## Attendees

- Hardware Team Members
- Teaching Assistant

## Agenda

1. Define hardware roadmap for function generator
2. Evaluate processing platforms
3. Assign research tasks and define investigation points

## Discussion

The meeting focused on defining the project roadmap and evaluating the hardware backbone for the proposed function generator. The discussion centered on selecting a processor capable of handling high-speed signal processing while maintaining a user-friendly interface.

## Hardware Platform Evaluation

1. Arduino: high accessibility but limited by processing speed for high-frequency waveforms.
2. ESP32: strong candidate due to 240 MHz dual-core performance.
3. Raspberry Pi Pico: robust PIO features but secondary to the others in this specific use case.
4. Strike Lite: prioritized for industrial stability and reliable timing.
5. PSoC: highly favored for its integrated programmable analog blocks.

## Final Shortlist for Research

After technical deliberation, the following three platforms were selected for deep-dive research:

- ESP32
- Strike Lite
- PSoC

## Preliminary System Architecture

The core signal chain was mapped out to identify potential bottlenecks. The diagram below illustrates the proposed flow from power input to signal output.

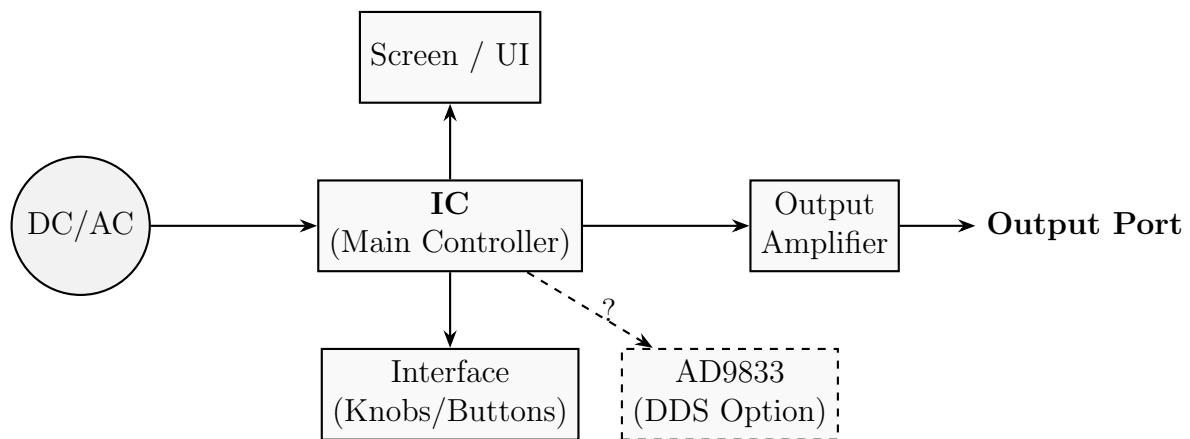


Figure 20: Preliminary System Architecture

## Research Methodology & Assignments

The Hardware, PCB, and related sub-teams have been divided into research groups. The focus for the next few days is as follows:

## Technical Comparison Matrix

| Feature            | ESP32         | Strike Lite              | PSoC               |
|--------------------|---------------|--------------------------|--------------------|
| Clock Speed        | 240 MHz       | High                     | Variable           |
| Analog Capability  | Internal DAC  | 8-bit External Dependent | Internal UDBs      |
| Complexity         | Low           | Medium                   | High               |
| Interfacing AD9833 | SPI Interface | SPI Interface            | Hardware-Level SPI |

### **Key Investigation Points**

- Signal Integrity: how the output amplifier handles noise from the switching regulators.
- DDS Integration: feasibility of using the AD9833 IC for high-precision waveform generation versus using internal MCU DACs.
- UI Latency: ensuring the screen refresh rate does not interfere with signal generation interrupts.

### **Action Items**

- Research teams will present their findings in the next meeting.
- Select the final IC and finalize the hardware requirement document (HRD) to begin schematic design.

# Minutes of Meeting

**Date:** 20/01/2026

**Topic:** Software Team (App-Based Control & Display Planning)

## Attendees

- Software Team Members

## Agenda

1. Decide communication method for control system
2. Confirm controller role and DDS integration
3. Plan display requirements

## Discussion

The meeting focused on app-based control and waveform display design.

### **ESP32 as Main Controller:**

- Built-in Wi-Fi and Bluetooth for wireless control
- Dual-core processor (up to 240 MHz) enabling real-time updates

### **Role of ESP32:**

- Handles Wi-Fi communication
- Receives user commands
- Computes frequency tuning values
- Configures the DDS chip via SPI
- Does not generate the waveform directly

### **DDS Requirement:**

- Recommended chip: AD9833
- Sine/Triangle up to  $\sim 1$  MHz
- Square up to  $\sim 10$  MHz

## Action Items

- Finalize OLED selection
- Confirm DDS integration
- Begin ESP32 firmware development

# Minutes of Meeting

**Date:** 20/01/2026

**Topic:** AutoCAD Modelling & Fabrication Basics

## Attendees

- Hardware Team Members
- CAD Team Members

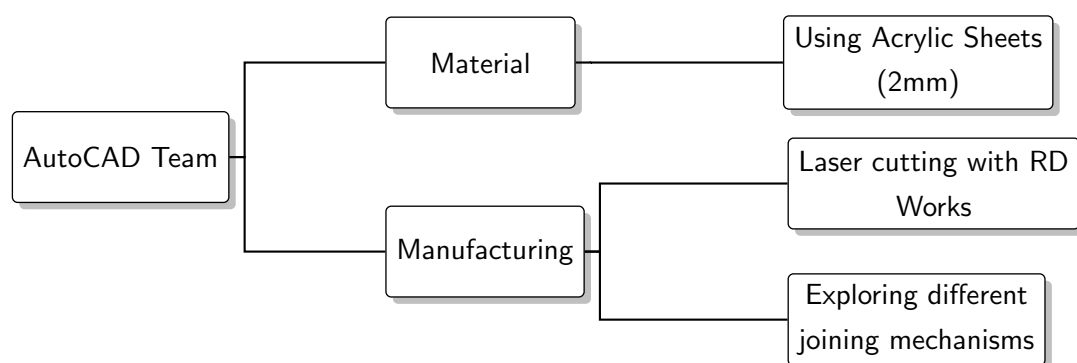
## Agenda

1. Introduce AutoCAD modelling workflow
2. Learn laser cutting setup and alignment
3. Practice acrylic handling and grooving techniques

## Discussion

### Acrylic Sheet Handling and Grooving Techniques

- Worked with 2 mm acrylic sheets and practiced folding techniques.
- Writing and marking on acrylic sheets were attempted to understand engraving and marking behavior.
- Grooving sketching techniques used for bending and folding acrylic materials were introduced.
- Emphasis was placed on groove depth, alignment, and their impact on structural integrity.



## Action Items

- Refine AutoCAD sketches and practice advanced dimensioning commands.
- Continue hands-on practice with RD Works and material processing.
- Explore different joining mechanisms and materials based on laser cutting compatibility.

# Minutes of Meeting

**Date:** 20/01/2026

**Topic:** Documentation Tools, Visualization & Report Workflow

## Attendees

- Documentation Team Members

## Agenda

1. Explore code-based mind mapping tools
2. Compile MoMs from different teams
3. Discuss report structure and photo PDF compression challenge

## Discussion

**Mind Mapping & Diagram Tools:** Team members explored `kroki.io` and tested multiple diagramming languages:

- TikZ, SVGbob, RackDiag, NwDiag, DBML, SeqDiag
- WireViz, C4 (PlantUML), Nomnoml, Mermaid

**MoM Compilation:** Progress and discussions from Hardware, Software, Research, and AutoCAD Modelling teams were noted for consolidation.

**Documentation Challenge:** Fitting 90 student photographs into a single PDF under 100 KB while keeping clarity acceptable was discussed.

## Action Items

- Finalize diagram tools for documentation
- Compile all MoMs into a unified document
- Optimize image compression strategy for the photo PDF
- Finalize report structure for upcoming submission

## Team 1 MOM: Software Team Discussion

|              |            |              |                          |
|--------------|------------|--------------|--------------------------|
| <b>Date:</b> | 27-01-2026 | <b>Team:</b> | Software Team            |
| <b>Day:</b>  | Day-3      | <b>Mode:</b> | Internal Team Discussion |

### Agenda

- Review progress of the Function Generator project.
- Define responsibilities of Software sub-teams.
- Discuss frontend, backend, and design workflows.
- Finalize sweep functionality logic and data flow.
- Coordinate software work with hardware (ESP32) team.

### Software Team Structure

The Software Team is divided into two major teams:

1. App Team
2. Display Team

#### App Team Sub-Divisions

The App Team is further divided into three sub-teams:

- Frontend Team
- Backend Team
- UI/UX Design Team

### Team Responsibilities & Progress

#### Frontend Team

**Responsibility:** Developing the client-side application interface for the function generator.

- **Controls Implemented:** Frequency and Amplitude adjustment; Wave type selection (Sine, Square, Triangular); Sweep functionality controls.

- **Research Conducted:** Validated WebSocket communication over traditional HTTPS for real-time, low latency, bidirectional communication.
- **Core Duties:** Sending control data to backend in JSON format; Real-time updates of waveform visualization; Maintaining flexible code structure for hardware adjustments.

## Backend Team

**Implementation:** Python and C++.

- **Core Duties:** Acts as the interface layer between frontend and hardware logic; Receiving JSON packets from the frontend; Parsing parameters: Frequency, Amplitude, Wave type, and Sweep settings.
- **Hardware Coordination:** JSON packets structured to extract wave requirements and control values; Logic coordinated with ESP32 generation constraints.

## UI/UX Design Team

**Design Philosophy:** Dark, instrument-inspired UI similar to professional lab equipment.

- **Implemented Elements:** Frequency Slider (10 Hz–1 MHz); Amplitude Slider (0–10 V); Wave type buttons; Real-time waveform display with grid.
- **Range Configuration:** Dynamic maximum frequency and amplitude input; Automatic slider range updates.
- **UX Goals:** Live visual feedback, user-friendly interaction, and consistent layout.

## Sweep Function - Finalized Features

### Sweep Modes

- **Linear Sweep:** Frequency increases by a fixed step (Hz). (Example: 10 Hz  $\rightarrow$  1010 Hz).
- **Logarithmic Sweep:** Frequency increases exponentially. Step defined as Steps per Decade. (Example: 10  $\rightarrow$  100  $\rightarrow$  1k).

## **Sweep Controls**

- Start Frequency & End Frequency; Step / Steps per Decade; Start Sweep button & Reset button.
- **Note:** Inputs disabled during active sweep to prevent conflicts; live status indicator active.

## **Software–Hardware Coordination**

- JSON packets generated by the frontend are aligned with ESP32 (Arduino) waveform generation code.
- Close coordination to match data formats and ensure correct parameter extraction.
- Minor code adjustments expected based on hardware timing and ESP32 limitations.

## **Decisions and Next Steps**

- **Decisions Taken:** WebSocket chosen for real-time communication; Step-based sweep logic finalized; UI design frozen.
- **Next Steps:** Integration testing, Hardware validation, and Display module improvements.

## Team 2 MOM: Hardware Team (ELP305)

**Date:** 27th January, 2026

### Component Selection & Research

- **Op-Amps:** LM7171, LM318, LM316, LM741, OPA 2863 researched (Robu.in, TI, Analog Devices, DigiKey).
- **Connectors:** Finalized banana connectors or PCT-213 for input voltage.
- **Priority:** LM318 required; current feedback amplifier identified.

### Technical Specifications

#### Device Output

- **Peak to peak max:** 20V; **Frequency range:** 2–2M Hz.
- **Waveform types:** Square, sine, triangle, noise, impulse, half square.
- **Resolution:** 0.004 Hz.

#### Power Requirements

- **Input:**  $\pm 15\text{V}$  DC; **ESP32:** 5V; **OLED Screen:** 5V.
- **Op-amp:** 12V (via buck converter).

#### Waveform IC: AD9833

- **Price:** 350; **Range:**  $\sim 0\text{--}1$  MHz (clean output); **Protocol:** SPI.
- **Assessment:** Cheapest/easiest SPI DDS; lacks AWG; needs filtering/amplification.

### OLED Display Module

- **Model:** 0.96 OLED Display Module (128x64); **Price:** 175 rs.
- **Link:** <https://robu.in/product/0-96-oled-display-module-spi-i2c-128x64-7-pin-blue>
- **Details:** No SMD components; easily connects with ESP32.

## Op-amp Analysis & Priority

- **LM7171:** Difficult to use (SMD: 448 rs, Through-hole: 760 rs).
- **OPA810:** Recommended candidate at 291 rs.
- **Priority List:** 1. OPA810, 2. LM7171.

## Additional Considerations

- **Controls:** Knobs, Numpad, Rotary knobs, and Push button (for power).
- **DAC:** No use.
- **Microcontroller:** ESP32 (Reason: Easy to code, required for OLED).
- **Extra Components:** BNC female coaxial cable.

## Member Responsibilities

- **AD9833 LTSpice/PCB:** Kalpesh, Manashvi; **Hardware Testing:** Sanidhya.
- **Input Ports:** Hardik; **Knob Signal Processing:** Aditeya.
- **CAD:** Nandini; **Power Distribution:** Suchir; **Proof Reading:** Arpan.

## Detailed Member Responsibilities

1. **AD9833 (Simulation Design):** Develop LTSpice models and handle PCB Designing — *Kalpesh, Manashvi*
2. **AD9833 (Hardware Testing):** Test using hardware libraries and verify output — *Sanidhya*
3. **Input Interface:** Determine port types and physical location for inputs — *Hardik*
4. **Control Signals:** Manage knob inputs and signal routing to the ESP32 — *Aditeya*
5. **CAD:** Mechanical design and modeling — *Nandini*
6. **Power Distribution System:** Design and implementation of power rails — *Suchir*
7. **Documentation:** Proofreading the requirements document — *Arpan*
8. **Microcontroller Justification:** Researching benefits (e.g., ease of coding and OLED compatibility) — *Suchir*

## Summary of Cost Analysis

| Component                | Specifications                      | Estimated Cost |
|--------------------------|-------------------------------------|----------------|
| Waveform IC (AD9833)     | 0–1 MHz range, SPI protocol         | 350            |
| OLED Display Module      | 0.96 inch, 128x64, SPI/I2C          | 175            |
| Op-Amp (OPA810)          | Priority candidate                  | 291            |
| Op-Amp (LM7171)          | SMD variant                         | 448            |
| Op-Amp (LM7171)          | Through-hole variant                | 760            |
| <b>Total (Tentative)</b> | <b>Sum of all listed candidates</b> | <b>2,024</b>   |

Table 5: Hardware Component Cost Estimates

*Note: The total cost is 2,024 (tentative) based on the inclusion of all listed op-amp variants; actual total will vary depending on the final op-amp selection.*

## Team 3 MOM: CAD Team

### Front Panel Components & Physical Layout

- **Components:** Rotary Knob  $\times$  2, BNC Connector  $\times$  2, LCD Screen, Numpad, On/Off Tactile Switch.
- **Numpad Dimensions:** 70 mm  $\times$  78 mm ( $4 \times 4$  buttons).
- **Design Tool:** RDWorks used for front panel layout and CAD design.
- **Modularity:** Modular front panel design proposed.
- **Mounting:** Rubber washers to be used to hold acrylic body panels.

### Mechanical Design & Fabrication Strategy

- **3D Printing:** Only front and back panels will be 3D printed; remaining body will be fabricated using acrylic sheets.
- **Manufacturing Approach:** Design part-by-part, but printing will be done in a single combined print.
- **Dimensions:** Discussion held on overall dimensions of the enclosure.
- **Front Panel Angle:** Wedge angle of front panel discussed and proposed as 60 degrees.
- **Prototyping:** Iteration over different wedge angles to be tested using thermocol sheet models.

### Ergonomics, Usability & Display Considerations

- **Button Placement:** Professor Kar requested reassessment of design; buttons should be placed on the top instead of the front to avoid pushing the function generator during operation.
- **Viewing Angle:** Display viewing angle should allow three students to view readings simultaneously.
- **Display Utilization:** Two columns of the display will be dedicated to show the waveform being generated, eliminating the need for separate LEDs and additional holes.
- **Durability:** Display-based waveform indication is more robust and avoids LED fusion issues.

- **Input Method:** Touch buttons suggested as a replacement for physical push buttons.
- **Numpad Usage:** TA advised avoiding the use of a numpad in the design.

## Thermal Management & Connectivity

- **Heat Dissipation:** Aluminium bottom plate proposed to dissipate heat instead of using cooling fans.
- **Connectivity:** Bluetooth can be used to sync the display output with mobile phones.



Figure 21: Thermocol prototype used to evaluate front panel wedge angles

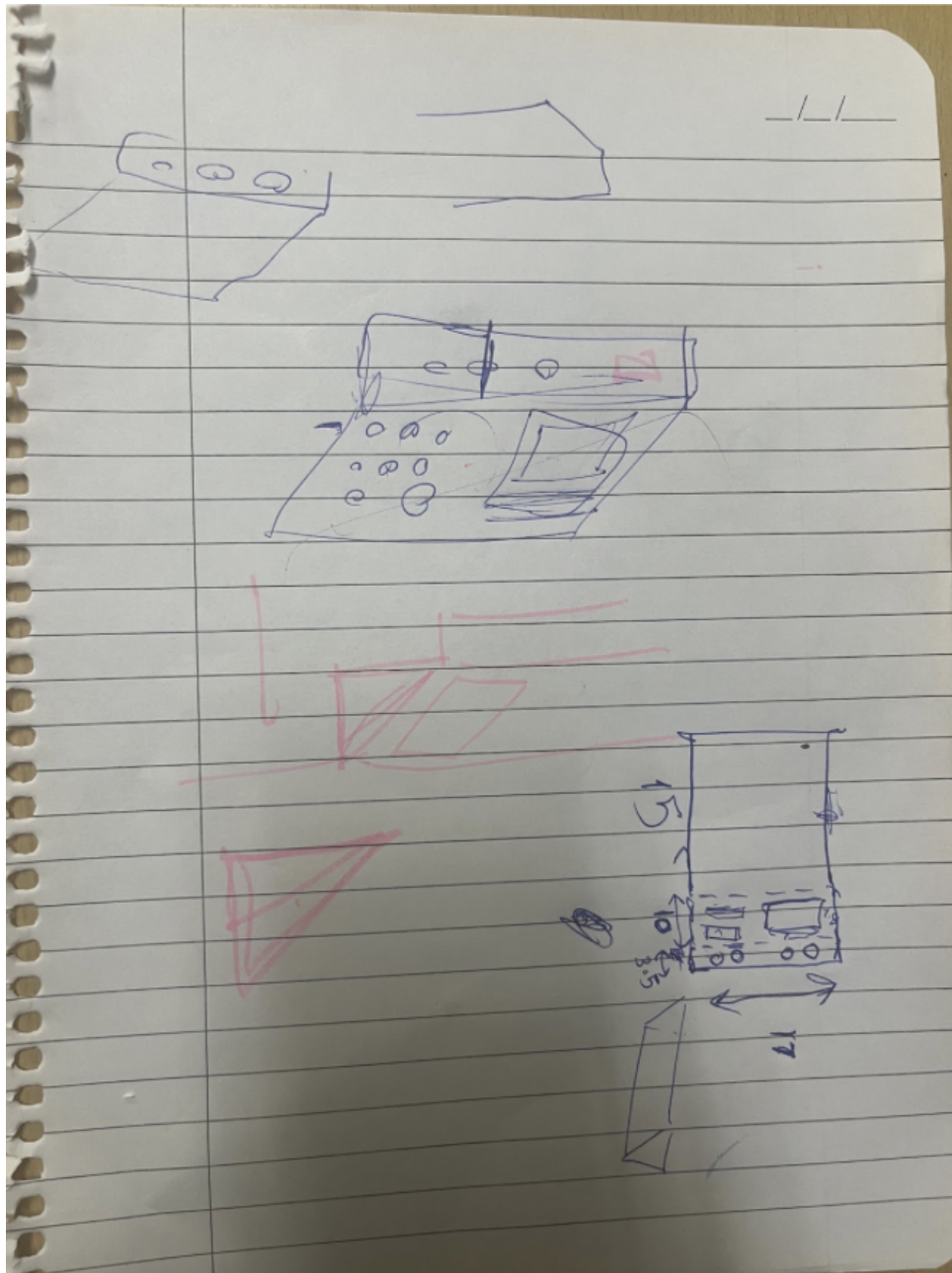


Figure 22: Initial discussion and sketches on enclosure dimensions

**Date:** 10/02/2026

**Topic:** Web App Backend Design

## Attendees

- Software Team Members

## Agenda

1. Discuss backend design for the web application
2. Review TA requirements for repository and documentation
3. Plan ESP32 web server simulation and code structure
4. Discuss GUI display layout and channel input configuration

## Discussion

The meeting focused on the backend architecture for the web application and development workflow planning.

### **Repository and Development Requirements:**

- TA requirements were reviewed.
- A **Git repository** was created and set to **private**.
- Git commits should be pushed **regularly over time** rather than on the same day.
- Code must be written to be **Doxygen compliant**.

### **ESP32 Web Server Planning:**

- Discussed **simulating the ESP32 web server**.
- Planned the **structure of the ESP32 code**.

### **Display GUI Design:**

- The **GUI design for the display** was discussed extensively.
- The **final button layout** for the interface was decided.

### **Input Channel Configuration:**

- The system was temporarily adapted for **single channel input**.
- After further discussion, the system was **adapted back to two-channel input**, which will be used in the final design.

**Date:** 10/02/2026

**Topic:** Hardware Interface Design Progress

## Attendees

- Hardware Team Members

## Agenda

1. Review hardware interface layout
2. Finalize control input components
3. Discuss display and channel control elements

## Discussion

The meeting focused on reviewing the hardware control interface layout and confirming the components used for user interaction with the function generator.

### **Control Interface Layout:**

- The overall hardware interface layout was reviewed, including the placement of the keypad, OLED display, additional buttons, and potentiometers.
- Soft wiring requirements for connecting the interface to the back-end control system were discussed.

### **Keypad and Input Controls:**

- A **numeric keypad (0–9)** will be used for direct entry of frequency and amplitude values.
- An **Enter key** will confirm and apply the entered values.
- **Left/Right navigation keys** will allow users to move between digits while editing parameters.

### **Parameter Adjustment Controls:**

- **x10 and /10 buttons** will allow quick adjustment of the order of magnitude of the input parameter.

- A **Wave Toggle button** will cycle between Sine, Square, and Triangle waveform outputs.
- An **Amplitude/Frequency toggle** will switch the editing focus between amplitude and frequency parameters.

#### **Channel Control and Display:**

- A **Channel Toggle** will determine which of the two output channels is displayed on the screen.
- Two **Channel On/Off buttons** will allow independent enabling or disabling of each channel.

#### **Analog Controls:**

- Two **offset control knobs (potentiometers)** will be used to adjust the DC offset of each channel.

## **MOM: CAD**

The CAD team designed and fabricated a transparent acrylic enclosure to house and protect the function generator. Panels were cut to size and shaped using a hot air gun to achieve the required form.

### **Key Design Features**

- Clear acrylic (PMMA) material for full equipment visibility
- Grid of square ventilation cutouts across all panels for airflow
- Angled front panel for ergonomic access
- Open-front design allowing full access to function generator controls

### **Fabrication Process**

- Acrylic sheets were cut according to the specified dimensions
- A hot air gun was used to bend and mold the panels to the required angles
- Panels were assembled to form the final enclosure box

### **Outcome**

The prototype enclosure was successfully built and its dimensions were verified to fit the target function generator unit.

### **Pending / Next Steps**

- Sand and smooth the acrylic edges
- Perform final dimensional verification
- Document and share the finalized CAD drawings

### **Prototype Image**

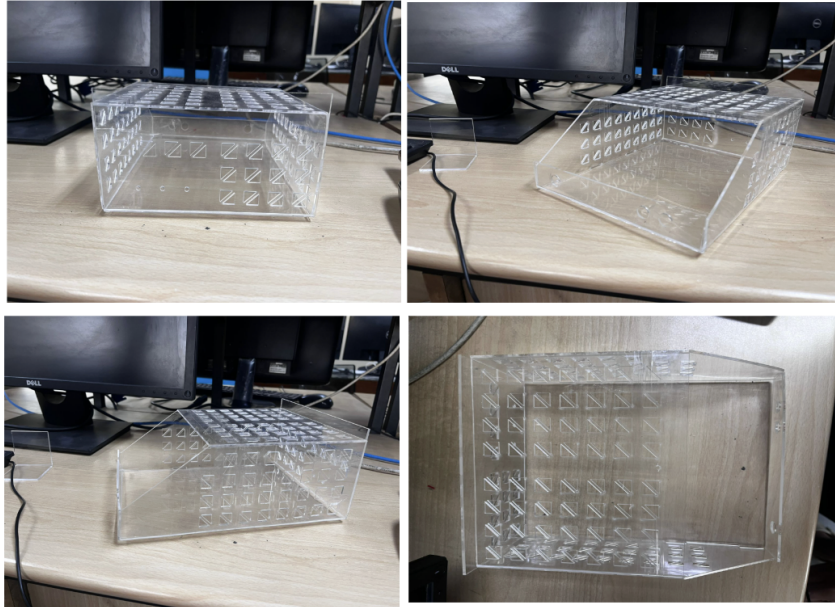


Figure 23: Prototype acrylic enclosure for the function generator